

目 录

1 基于 face-api.js 的浏览器端人脸分析实验	1
1.1 实验定位与章节边界	1
1.2 项目结构与资源组织	2
1.2.1 静态项目结构	2
1.2.2 模型权重与本地加载	2
1.2.3 页面布局与显示容器	3
1.3 脚本加载与初始化流程	3
1.3.1 defer 脚本顺序	3
1.3.2 模型加载的并行等待	4
1.4 图片输入处理流程	5
1.4.1 从本地文件到图像对象	5
1.4.2 检测结果的尺寸映射	5
1.5 表情识别模块分析	6
1.5.1 模型调用链	6
1.5.2 结果排序与概率展示	6
1.5.3 实验现象与局限	7
1.6 年龄与性别估计模块分析	7
1.6.1 模型调用链与输出字段	7
1.6.2 Canvas 标注与结果卡片	7
1.6.3 结果解释与伦理边界	7
1.7 摄像头实时追踪模块分析	8
1.7.1 摄像头授权与视频流绑定	8
1.7.2 周期检测与画布同步	9
1.7.3 结果节流与状态管理	9
1.8 实验复现步骤与观察记录	9
1.8.1 页面访问与资源检查	9
1.8.2 HTML 结构与脚本变量对应	10
1.8.3 建议记录的实验现象	11
1.9 异常场景与边界条件	11
1.9.1 模型文件加载失败	11
1.9.2 输入为空或格式异常	12

1.9.3	摄像头权限与安全上下文	12
1.10	评价指标与实验扩展	12
1.10.1	检测类指标	12
1.10.2	属性估计指标	12
1.10.3	实时性能指标	13
1.11	综合实验结果与分析	13
1.11.1	功能完整性	13
1.11.2	性能影响因素	14
1.11.3	可靠性与数据质量	14
1.12	隐私、安全与改进方向	14
1.13	本章小结	15
2	dlib 开源人脸识别实例分析	15
2.1	分析目的与章节边界	15
2.2	示例入口与数据目录组织	16
2.2.1	命令行输入与顶层目录	16
2.2.2	训练数据与参考样本的区别	17
2.3	mini-batch 采样机制	18
2.3.1	为什么度量学习依赖 batch 组成	18
2.3.2	随机选择与标签生成	18
2.3.3	数据增强与尺寸约束	19
2.4	网络结构与 128 维描述子	19
2.4.1	ResNet 主体结构	19
2.4.2	训练网络与测试网络	20
2.4.3	128 维输出向量	20
2.5	loss_metric 损失函数	20
2.5.1	距离关系作为训练目标	20
2.5.2	嵌入空间的直观理解	21
2.6	训练器与优化过程	21
2.6.1	训练器配置	21
2.6.2	学习率停止条件	21
2.7	主函数训练流程拆解	22
2.7.1	参数检查与数据读入	22

2.7.2	训练对象初始化	22
2.7.3	生产者消费者训练循环	22
2.8	多线程数据加载	23
2.8.1	为什么需要异步加载	23
2.8.2	线程数量与资源平衡	23
2.9	训练数据准备与质量控制	23
2.9.1	身份数量与样本数量	23
2.9.2	图片质量与预处理	24
2.9.3	数据增强的作用边界	24
2.10	模型保存与训练后验证	25
2.10.1	模型清理与序列化	25
2.10.2	训练后验证流程	25
2.11	距离阈值与开放集识别	25
2.11.1	阈值来源	25
2.11.2	与闭集分类的区别	26
2.12	验证指标与阈值选择	26
2.12.1	num_right 与 num_wrong 的含义	26
2.12.2	正负样本对的数量	26
2.12.3	阈值调节的取舍	27
2.13	与浏览器实验的对应关系	27
2.14	实验启示与局限	28
2.15	本章小结	28
参考文献		29
附录 人脸分析实验关键代码		30
	页面入口与展示容器	30
	图片表情识别脚本	32
	年龄与性别估计脚本	36
	摄像头实时追踪脚本	39
	开源人脸识别模型训练代码	45

1 基于 face-api.js 的浏览器端人脸分析实验

1.1 实验定位与章节边界

本章分析的是 `code/face-recognition/` 目录中的浏览器端人脸分析实验。该项目以 HTML、CSS 和 JavaScript 静态资源为主体，通过 `face-api.js`[1] 在浏览器中加载预训练模型，并在本地完成图片表情识别、图片年龄与性别估计以及摄像头实时人脸追踪。它的重点不是训练新的神经网络，而是观察一个已经训练好的模型体系如何被前端页面组织、加载、调用和展示。

为了避免与下一章内容混淆，本章只把训练原理作为必要背景来说明：页面中的 `models/` 目录保存的是预训练权重，`public/img/labeled_images` 中的人物图片更接近“参考样本”或“注册样本”，并不会在浏览器端更新神经网络参数。人脸描述子、距离阈值和度量学习的训练机制将在第 2 章结合 `dlib` 示例展开。本章关注的是推理阶段，即输入如何进入模型、模型如何返回结构化结果、页面如何把结果画出来。

人脸分析是计算机视觉在人机交互、身份认证、内容理解和智能终端中的典型任务。传统系统常把模型部署在服务器端，客户端只负责采集图像、上传数据和显示结果。这种方式便于集中管理模型，但会带来网络依赖、数据传输、服务部署和延迟控制等问题。随着 `TensorFlow.js` 等前端推理框架的发展，浏览器不仅能显示页面，也能成为模型推理运行环境。对于课程实验而言，这种部署方式具有明显优势：运行入口直观、依赖环境轻、实验过程容易展示，也便于在报告中把“页面结构”和“识别流程”对应起来分析。

本实验选择三个页面作为观察对象：`face_expression_recognition.html` 用于图片表情识别，`age_gender_estimation.html` 用于年龄与性别估计，`webcam_face_tracking.html` 用于摄像头实时追踪。三个页面共享同一个模型目录和前端资源目录，但各自加载不同业务脚本。通过对它们进行对比，可以看出浏览器端人脸分析系统的一般流程：先加载模型，再取得输入，再执行模型推理，最后把检测框、关键点、表情或属性结果渲染到界面上。

本章的实验目标包括四个方面。第一，梳理静态网页项目中页面、样式、脚本、模型权重和示例资源之间的组织关系；第二，分析图片输入与摄像头视频流输入在前端实现上的差异；第三，分别说明表情识别、年龄性别估计和实时追踪三个功能模块的实现逻辑；第四，总结浏览器端推理在性能、稳定性、隐私和实验复现方面的优点与局限。

1.2 项目结构与资源组织

1.2.1 静态项目结构

该项目没有后端服务,也没有复杂的构建入口。页面入口直接位于 `code/face-recognition/` 根目录,业务脚本位于 `dist/`,模型文件位于 `models/`,示例图片、人物注册图片和演示视频位于 `public/`。从实验复现角度看,这种结构的好处是入口清晰:只要浏览器能够访问静态文件,就可以加载页面并触发识别逻辑;对于摄像头页面,还需要浏览器允许页面访问本机摄像头。

项目中的核心库文件是 `dist/face-api.min.js`。它是 `face-api.js` 的浏览器端构建文件,向页面暴露全局对象。三个实验页面并不直接实现底层神经网络,而是调用这个对象提供的高层接口,例如模型加载接口、检测接口、关键点接口、表情接口和绘图接口。业务脚本则负责把这些接口连接成完整实验流程。

页面、脚本和功能之间的对应关系如表 1.1 所示。

表 1.1 浏览器端人脸分析实验页面与脚本对应关系

页面文件	业务脚本	主要功能
<code>face_expression_recognition.html</code>	<code>dist/expression_script.js</code>	图片表情识别
<code>age_gender_estimation.html</code>	<code>dist/estimation_script.js</code>	年龄与性别估计
<code>webcam_face_tracking.html</code>	<code>dist/webcam_script.js</code>	摄像头实时追踪



图 1.1 浏览器端人脸分析项目目录结构占位图

1.2.2 模型权重与本地加载

`models/` 目录中保存了多个预训练模型权重,包括 Tiny Face Detector、SSD MobileNetV1、68 点人脸关键点模型、表情识别模型、年龄性别估计模型和人脸识别模

型等。业务脚本中的 均设置为 ，说明当前实验版本从本地目录加载模型，而不是依赖远程 CDN 或 GitHub Raw URL。

本地模型加载对课程报告很重要。首先，它减少了外部网络波动对实验复现的影响。其次，它使报告中的路径说明与实际项目文件保持一致，读者可以直接在仓库中找到模型、脚本和示例资源。最后，本地加载也更容易观察模型文件数量和页面请求过程，有助于理解前端深度学习应用并不是只有一个 HTML 页面，而是由页面、模型权重、脚本和媒体资源共同组成。

需要注意的是，本地保存权重不等于本地训练模型。浏览器页面运行时只通过把权重读入内存，然后执行前向推理。无论用户上传多少图片，模型参数都不会被更新。这个边界决定了本章的分析口径：页面展示的是“如何使用模型”，不是“如何训练模型”。

1.2.3 页面布局与显示容器

三个页面采用相似的界面组织方式：顶部为导航区域，中间主体为左右分栏。左侧通常放置文件上传控件、摄像头状态、识别结果卡片和空状态提示；右侧放置图片或视频展示区域，并在媒体元素上叠加 Canvas 画布。这样的布局适合实验观察，因为图像标注和结构化结果可以并排出现。用户既能看到检测框是否对准人脸，也能读取模型输出的表情概率、年龄估计、性别置信度等信息。

从职责分工看，HTML 负责定义页面结构，CSS 负责布局和视觉样式，JavaScript 负责模型加载、输入处理、推理调用和结果渲染。页面本身不包含复杂算法，算法能力主要来自 `face-api.js` 和预训练模型。业务脚本则处在连接层：它把 DOM 输入转成模型可处理的媒体对象，把模型输出转成 Canvas 标注和结果卡片。

1.3 脚本加载与初始化流程

1.3.1 defer 脚本顺序

三个页面都在 `head` 中先引入样式文件，再使用 `defer` 引入脚本。脚本顺序保持为先加载 `dist/face-api.min.js`，再加载当前页面对应的业务脚本。这一顺序是稳定运行的前提，因为业务脚本会直接访问全局对象。如果业务脚本先执行，就会出现之类的错误。

`defer` 的作用是让浏览器在解析 HTML 时并行下载脚本，但等到文档结构解析完成后再按页面中的顺序执行。这样，业务脚本执行时既能访问已经加载好的对象，也能通过 找到上传控件、结果容器和视频元素。对于静态实验页面来说，这是一种简洁稳定的初始化方式，不需要额外把所有逻辑包在复杂的 DOM ready 回调中。

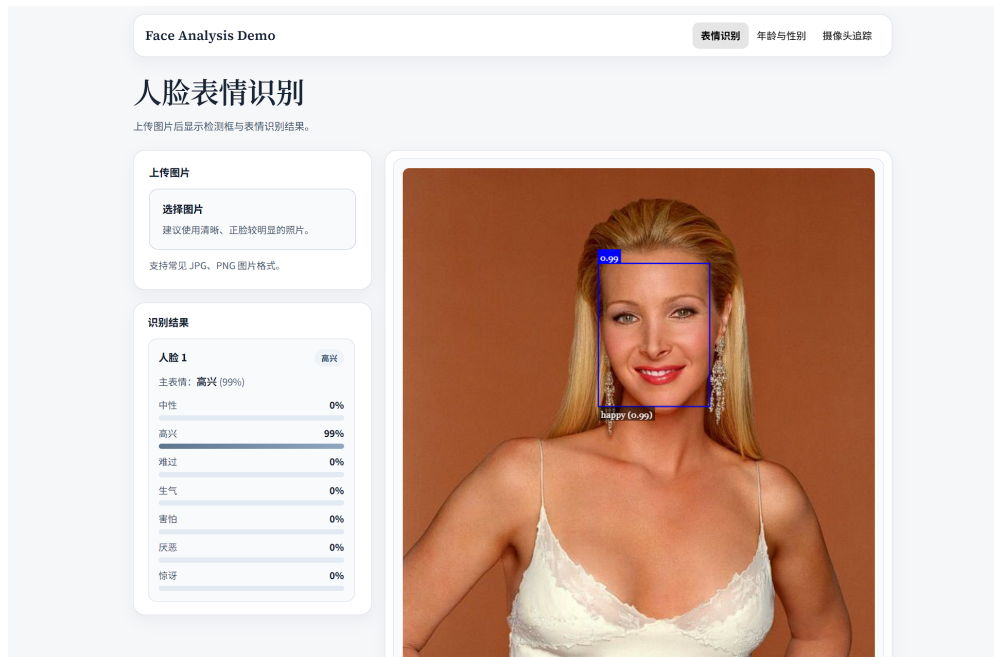


图 1.2 表情识别页面运行截图

页面底部还可能引入 jQuery、Popper、Bootstrap 和 MDB 等脚本。这些库主要承担界面组件和样式交互职责,不直接参与模型推理。实验的核心识别链路由 `face-api.js` 与三个业务脚本完成,其他前端库只改善页面展示体验。

1.3.2 模型加载的并行等待

业务脚本使用 并行加载所需模型。表情识别脚本加载 Tiny Face Detector、68 点关键点、人脸识别、表情识别和 SSD MobileNetV1; 年龄性别估计脚本加载 Tiny Face Detector、68 点关键点、人脸识别、年龄性别估计和 SSD MobileNetV1; 摄像头脚本加载 Tiny Face Detector、68 点关键点、人脸识别和表情识别模型。只有当这些 Promise 全部完成后,脚本才进入后续的 或 流程。

这种写法体现了前端模型应用的一个基本原则: 在用户交互之前先确保模型可用。否则用户可能已经上传图片,但模型权重还没有加载完成,页面就会进入不确定状态。脚本通过等待模型加载完成后再注册上传监听器或请求摄像头,避免了“输入已经到达但模型尚未准备好”的竞态问题。

不同页面加载的模型略有冗余。例如图片页面加载了,但当前展示重点不一定使用身份匹配结果。这种设计可能是为了复用脚本模板或保留扩展空间。就实验报告而言,可以把它理解为典型的人脸分析流水线: 检测模型先找到人脸,关键点模型定位面部结构,描述子模型提取特征,属性模型输出表情、年龄或性别。

1.4 图片输入处理流程

1.4.1 从本地文件到图像对象

图片类页面通过 `input[type=file]` 接收用户上传的 JPG、PNG 等常见图片格式。业务脚本监听上传控件的 事件。如果用户没有选择文件，脚本直接返回；如果存在文件，页面先渲染“正在识别”的空状态提示，然后清除上一张图片 and 上一张画布，避免旧结果与新结果叠在一起。

上传文件并不能直接送入模型。脚本调用，将本地文件转换为浏览器中的图片对象，再把图片插入展示容器。随后，脚本调用 创建覆盖画布，使模型标注能够叠加在图片上。这里的 Canvas 是结果可视化的关键：检测框、表情标签、年龄性别标签都不是写在原图上，而是画在覆盖层中。

脚本根据图片的显示尺寸构造，再调用 使画布尺寸与图片一致。这个步骤容易被忽略，但它直接影响标注位置。如果图片显示尺寸与 Canvas 尺寸不一致，模型输出框即使在原始坐标中正确，绘制到页面上也可能发生偏移。

1.4.2 检测结果的尺寸映射

模型推理完成后，脚本调用 将检测结果映射到当前显示尺寸。浏览器页面中图片可能受到 CSS 缩放影响，因此模型输出坐标与页面显示坐标不一定完全一致。通过尺寸映射，检测框和属性标签才能准确叠加到用户看到的图片位置上。

图片页面的通用流程可以概括为五步：第一，监听文件上传事件；第二，将文件转换为图片对象并插入页面；第三，创建与图片匹配的 Canvas 覆盖层；第四，调用人脸检测和属性识别接口；第五，将结果映射到显示尺寸，并同时绘制到 Canvas 与渲染到侧边栏。这个流程也说明，浏览器端实验并不是单纯调用一个识别函数，而是包含输入转换、异步等待、尺寸同步、结果排序和界面反馈等多个工程环节。



图 1.3 图片输入与画布标注流程占位图

1.5 表情识别模块分析

1.5.1 模型调用链

表情识别由 `dist/expression_script.js` 实现。脚本在图片上传后调用如下链式接口：先执行 检测所有人脸，再串联 获取 68 点关键点，接着调用 提取人脸描述子，最后调用 得到表情分类概率。最终返回的每个人脸结果中包含检测框、关键点、描述子和表情分数。

表情模型输出七类表情概率：、、、、、、。业务脚本将它们映射为中文标签：中性、高兴、难过、生气、害怕、厌恶和惊讶。映射后的标签更适合课程报告和中文界面阅读。

1.5.2 结果排序与概率展示

当图片中存在多张人脸时，脚本并不直接使用模型返回顺序，而是根据检测框的横坐标进行排序，使结果卡片顺序与图片中从左到右的人脸位置一致。这是一个很小但很实用的交互细节。对于多人脸图片，如果侧边栏结果顺序与图像位置不一致，读者很难判断某个概率属于哪张脸。

每张人脸的结果卡片会先显示主表情，即七类概率中分数最高的一项。随后，脚本遍历所有表情类别，用百分比和进度条展示完整概率分布。这样做比只显示一个标签更有解释价值。例如模型可能判断“高兴”为 54%，“中性”为 38%，这说明结果并不是绝对确定，而是在多个类别之间存在竞争。对于课程实验而言，展示概率分布有助于理解模型输出的置信度。

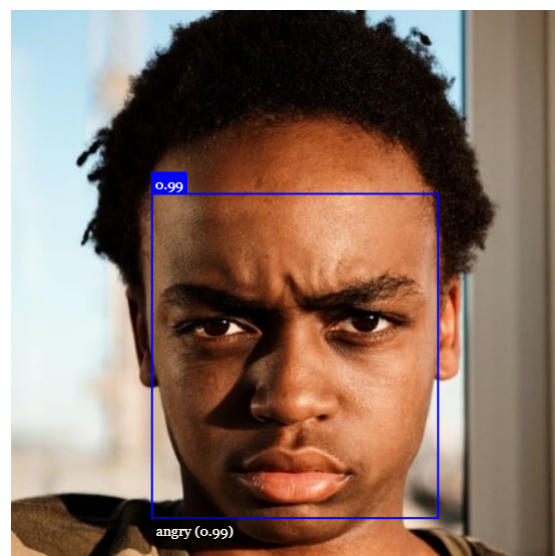
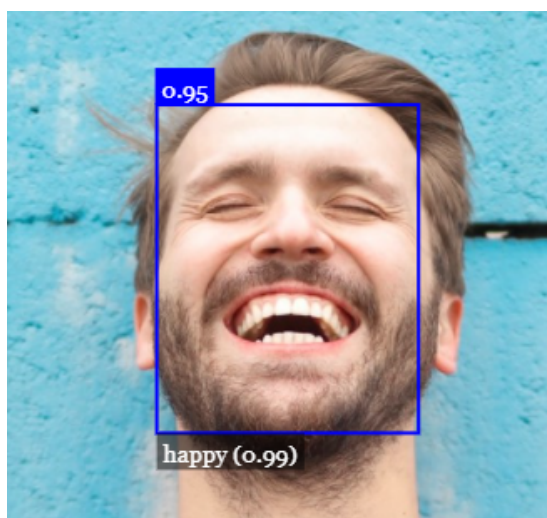


图 1.4 表情识别示例结果

1.5.3 实验现象与局限

表情识别对人脸姿态、遮挡、清晰度和表情幅度比较敏感。正脸、光照均匀、面部无遮挡的图片通常更容易得到稳定结果；侧脸、低分辨率、强背光或面部被手、口罩、头发遮挡时，模型可能无法检测到人脸，或者检测到了人脸但表情概率分布不稳定。表情本身也具有主观性，同一张图片可能同时包含“中性”和“高兴”的特征，因此输出应理解为模型概率估计，而不是人工情绪判断。

从界面反馈看，脚本为“正在识别”和“未检测到人脸”设置了空状态提示。这些状态能降低用户等待时的不确定感，也使页面更接近完整应用，而不是只在控制台打印结果的演示代码。课程报告中记录这些状态，有助于说明实验系统不仅关注模型调用，也关注用户能否理解当前运行阶段。

1.6 年龄与性别估计模块分析

1.6.1 模型调用链与输出字段

年龄与性别估计由 `dist/estimation_script.js` 实现。它与表情识别页面共享类似的图片上传、Canvas 创建和尺寸映射流程，但推理链路末尾换成。模型输出主要包括检测框、估计年龄、性别标签和性别置信度。脚本将 和 映射为“男性”和“女性”，并将年龄四舍五入后以“岁”为单位展示。

该模块体现了多任务人脸属性估计的基本思路。人脸检测模型先确定候选区域，关键点模型提供面部结构信息，描述子模型提取可用于表示身份或面部特征的向量，年龄性别模型再在检测到的人脸区域上输出外观属性。虽然脚本使用链式接口写得很短，但背后实际包含多个模型的顺序协作。

1.6.2 Canvas 标注与结果卡片

年龄性别页面使用 绘制检测框，然后为每张人脸创建，将年龄和性别字符串作为框标签叠加到图像上。同时，侧边栏结果卡片会展示年龄估计、性别判断和性别置信度。图像标注适合快速定位，侧边栏适合读取更完整的结构化结果。

1.6.3 结果解释与伦理边界

年龄和性别估计结果必须谨慎解释。模型看到的是图像中的外观特征，而不是人的真实身份信息。年龄估计会受到妆容、光照、表情、拍摄角度和图像质量影响；性别估计则涉及更敏感的个人属性，也可能受到训练数据分布和社会偏差影响。因此，本实验中的结果应称为“模型估计”或“外观属性预测”，不应被表述为真实年龄或真实性别。

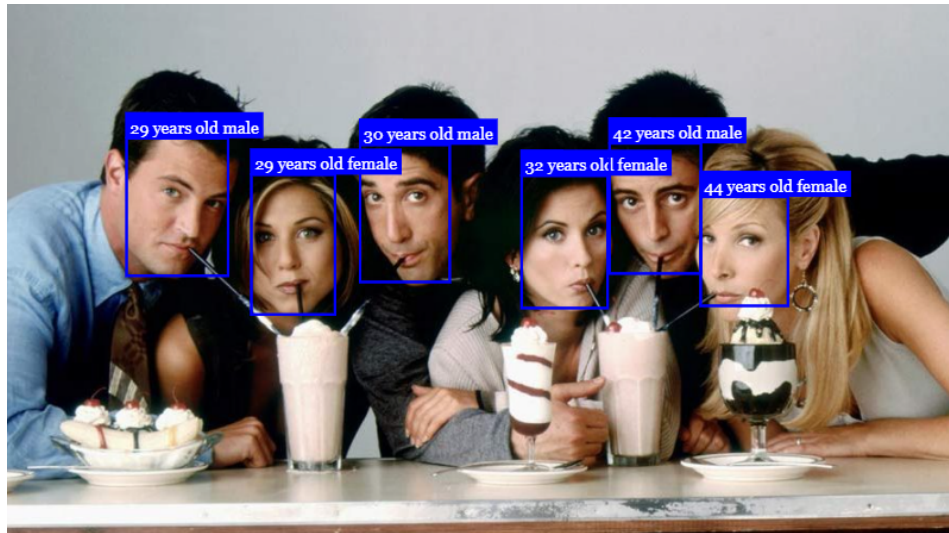


图 1.5 年龄与性别估计示例结果

在实际应用中，这类属性输出不适合直接用于身份、资格、风险、权限等高影响决策。课程实验可以用它展示模型能力和局限，但报告中应明确说明其不确定性。这样的表述既符合技术事实，也能避免把演示模型包装成可靠判断工具。

此处放置年龄与性别估计页面运行截图。
建议文件名：img/age_gender_page_runtime.png。
截图内容建议包含上传控件、右侧图片标注和左侧结果卡片。

图 1.6 年龄与性别估计页面运行截图占位图

1.7 摄像头实时追踪模块分析

1.7.1 摄像头授权与视频流绑定

摄像头实时追踪由 `dist/webcam_script.js` 实现。页面启动后先显示“等待授权”状态，并检查浏览器是否支持媒体采集接口。脚本优先使用 `getUserMedia`，同时保留旧版 `webkitUserMedia` 和 `msUserMedia` 的兼容写法。如果浏览器不支持这些接口，页面会显示“摄像头不可用”；如果用户拒绝授权或设备不可用，页面会显示“访问失败”。

成功获取摄像头视频流后，脚本将 `video` 元素赋值给 `videoElement`。此时浏览器负责把摄像头画面播放到页面中的 `video` 元素。只有当视频触发 `loadedmetadata` 事件后，脚本才创建覆盖画布并启动周期

性识别。这种顺序很合理，因为视频尺寸和播放状态尚未确定时，提前创建画布容易导致尺寸不匹配。

1.7.2 周期检测与画布同步

实时追踪的核心是一个 100 ms 间隔的。每次定时器触发时，脚本读取当前视频帧，调用 检测人脸，再串联关键点定位和表情识别。推理完成后，脚本将结果映射到当前显示尺寸，清空上一帧画布内容，并重新绘制检测框、68 点关键点和表情标签。

与图片页面相比，摄像头页面更强调连续性和性能。图片页面只需要处理一次上传图片，摄像头页面则不断处理视频帧。如果每帧都执行复杂模型，低性能设备可能出现卡顿。因此脚本选择 Tiny Face Detector 作为实时检测器，并把识别间隔设置为 100 ms，相当于理论上每秒最多处理约 10 次。这个频率不是视频帧率本身，而是模型推理和标注更新频率。

脚本中的 会根据展示容器尺寸同步 Canvas 大小，窗口尺寸变化时也会重新匹配。这个函数解决了实时页面常见的标注错位问题：视频画面可能因为响应式布局变化而改变显示尺寸，如果 Canvas 不跟随更新，检测框就会偏离人脸位置。

1.7.3 结果节流与状态管理

摄像头页面不仅要绘制画布，还要更新侧边栏结果。高频更新 DOM 会带来额外开销，因此脚本设置了，并用 缓存上一次结果。当本次结果与上次相同，或者距离上次渲染时间不足 280 ms 时，脚本会跳过侧边栏更新。这样可以减少重复 DOM 写入，让页面把更多资源留给视频显示和模型推理。

脚本还监听视频的 和 事件，及时清除检测定时器并更新页面状态。这个细节说明实时推理任务需要生命周期管理：当视频已经暂停，如果定时器仍在后台运行，就会浪费计算资源，也可能造成用户误以为识别还在进行。

1.8 实验复现步骤与观察记录

1.8.1 页面访问与资源检查

复现实验时，首先需要确认项目目录中的静态资源完整。三个 HTML 页面必须能够访问 css/、js/、dist/、models/ 和 public/ 等目录。若页面打开后只有布局而没有识别能力，通常需要检查 dist/face-api.min.js 是否加载成功；若页面能够显示但长时间无法识别，通常需要检查 models/ 中的权重文件是否存在，以及浏览器控制台是否出现模型加载失败信息。

此处放置摄像头实时追踪截图或 GIF 截帧。
建议文件名：`img/webcam_tracking_runtime.png`。
截图内容建议包含视频画面、检测框、关键点和侧边栏实时结果。

图 1.7 摄像头实时追踪运行效果占位图

对于图片类页面，实验观察可以从三个层次记录。第一层是页面结构是否正常，包括导航栏、上传控件、结果区域和图片展示区域是否出现。第二层是模型初始化是否正常，即上传图片后是否能进入“正在识别”状态并最终绘制结果。第三层是识别结果是否合理，包括检测框是否覆盖人脸、侧边栏结果数量是否与图中人脸数量一致、概率或置信度是否以可读形式展示。

对于摄像头页面，复现步骤还需要额外关注浏览器权限。页面必须在浏览器允许摄像头访问的环境下运行；如果浏览器阻止摄像头权限，页面不会进入实时识别阶段。摄像头页面的观察重点包括状态卡片是否从“等待授权”变为“正在连接”或“摄像头已连接”，视频画面是否正常播放，检测框和关键点是否随人脸移动更新，暂停视频后定时器是否停止。

1.8.2 HTML 结构与脚本变量对应

三个页面的 HTML 结构与脚本变量之间存在明确对应关系。表情识别和年龄性别估计页面都包含 `id="upload"` 的文件输入控件，脚本通过 `upload` 获取它；右侧结果区域使用 `id="analysis-stage"`，脚本把图片和 Canvas 都插入这个区域；侧边栏结果则分别使用 `id="expression-results"` 和 `id="estimation-results"`。

摄像头页面的关键元素不同。视频元素使用 `id="video"`，脚本把摄像头流绑定到该元素；画布覆盖层插入 `id="tracking-stage"` 对应的容器；摄像头状态写入 `id="webcam-status"`；实时识别结果写入 `id="webcam-results"`。这种 DOM 命名方式让页面结构和业务脚本能够直接对应，便于阅读和调试。

从报告写作角度看，记录这些 DOM 对应关系有两个作用。第一，它能证明实验分析来自实际代码，而不是只描述模型概念。第二，它能解释为什么 HTML 中的元素命名会影响脚本运行。如果某个元素的 `id` 被改动，而脚本仍按旧名称查找，页

面就会在初始化阶段出现空引用错误。

此处放置 HTML 关键 DOM 与脚本变量对应截图。
建议文件名：`img/face_api_dom_mapping.png`。
截图内容建议左侧显示 HTML 中的 `id`，右侧显示脚本中的 `getElementById`。

图 1.8 页面 DOM 与业务脚本变量对应占位图

1.8.3 建议记录的实验现象

为了让报告更像完整实验，而不是只像代码说明，可以在复现时记录若干具体实验现象。表情识别页面可以分别上传单人正脸、多人合照、侧脸、模糊图片和夸张表情图片，观察检测数量和表情概率变化。年龄性别估计页面可以记录不同年龄段、不同光照和不同姿态图片下的估计波动。摄像头页面可以记录正脸静止、头部转动、多人进入画面和光照变化时的实时标注稳定性。

这些观察不一定需要形成严格统计实验，但应体现输入条件与输出结果之间的关系。例如，当人脸较小或模糊时，检测框可能消失；当表情不明显时，中性概率可能占主导；当摄像头画面快速移动时，关键点可能出现短暂抖动。把这些现象写入报告，可以避免只呈现成功案例，也能体现对模型局限的理解。

1.9 异常场景与边界条件

1.9.1 模型文件加载失败

浏览器端实验最常见的失败场景之一是模型文件加载失败。如果 `MODEL_URL` 指向错误目录，或者 `models/` 中缺少权重文件，将无法进入后续流程。用户看到的现象可能是页面可以打开，但上传图片后没有反应，或者摄像头页面一直停留在初始状态。此时应优先检查浏览器控制台和网络请求列表。

由于模型加载是异步过程，页面最好提供加载失败提示。当前脚本主要处理正常加载路径，对于模型加载异常的用户反馈仍有改进空间。课程报告可以把这一点列为后续优化方向：在 后补充 分支，提示“模型资源加载失败，请检查 `models` 目录或网络路径”。

1.9.2 输入为空或格式异常

图片页面已经对空文件选择做了简单保护：当 为 0 时，脚本直接返回。由于上传控件使用 `accept="image/*"`，浏览器会尽量限制用户选择非图片文件。但这种限制并不是严格安全边界，实际应用仍应在脚本中处理图片解码失败的异常。

如果图片过大，浏览器端推理可能明显变慢，甚至造成页面短暂无响应。报告中可以说明，课程实验适合使用清晰但尺寸适中的图片。实际应用中可以在前端对图片进行压缩、缩放或限制最大尺寸，再送入模型推理，从而减少计算开销。

1.9.3 摄像头权限与安全上下文

摄像头页面还可能受到浏览器安全策略影响。现代浏览器通常要求摄像头访问发生在安全上下文中，例如 HTTPS 或本机 localhost 环境。若直接以某些文件协议打开页面，摄像头接口可能不可用。脚本虽然兼容新旧 接口，但无法绕过浏览器权限策略。

用户拒绝授权、设备被其他程序占用、摄像头不存在或浏览器不支持媒体采集接口，都会导致实时追踪失败。当前脚本会显示“访问失败”或“不支持”等状态，这对于实验页面已经比较友好。后续若扩展为正式应用，还可以增加更具体的错误分类，例如权限拒绝、设备未找到、设备忙碌和安全上下文不满足。

1.10 评价指标与实验扩展

1.10.1 检测类指标

如果需要把本实验从功能展示扩展为定量分析，可以首先设计检测类指标。对于人脸检测，可统计测试图片总数、真实人脸总数、成功检测人脸数、漏检数和误检数。检测成功率可以粗略表示为成功检测人脸数与真实人脸总数之比。对于多人脸图片，还可以记录检测框位置是否与真实人脸区域基本重合。

课程报告不一定需要构建大规模数据集，但可以选取若干代表性样例：正脸单人、多人合照、侧脸、强光、弱光、遮挡、低清晰度等。通过这些样例记录模型表现，可以让实验结论更有依据。例如，报告可以说明“在正脸清晰图片上识别较稳定，在遮挡和侧脸场景中容易漏检”。

1.10.2 属性估计指标

对于表情识别，评价可以观察主表情是否符合人工直观判断，以及概率分布是否集中。对于年龄估计，可以记录模型估计值与人工标注年龄的大致偏差，但要注意隐私和数据来源。对于性别估计，不建议把结果作为强标签评价，更适合讨论模

型输出的不确定性和潜在偏差。

属性估计的结果不应只追求“准确率”一个数字。表情、年龄和性别都涉及主观性、社会属性和数据偏差。更合理的写法是结合置信度、输入质量和适用边界进行分析。尤其在课程报告中，应把这些输出理解为模型能力展示，而不是可靠身份判断。

1.10.3 实时性能指标

摄像头页面可以记录实时性能指标，例如模型更新间隔、页面体感延迟、检测框抖动情况和设备资源占用。当前脚本的检测间隔设置为 100 ms，但实际推理耗时可能超过 100 ms；当推理时间较长时，页面无法真正达到每秒 10 次稳定更新。报告中可以说明，定时器间隔是目标频率，实际频率还取决于设备性能和模型耗时。

如果后续要优化实时性能，可以考虑降低视频显示分辨率、减少绘制内容、只绘制检测框不绘制关键点、延长检测间隔，或者使用更轻量的检测参数。优化方向需要在速度、稳定性和结果丰富度之间取舍。

1.11 综合实验结果与分析

1.11.1 功能完整性

从功能完整性看，该项目完成了三个典型的人脸分析任务。表情识别页面能够在上传图片后检测多张人脸，并输出每张人脸的主表情和七类表情概率；年龄与性别估计页面能够输出检测框、估计年龄、性别标签和置信度；摄像头追踪页面能够在获得权限后持续处理视频帧，并实时叠加检测框、关键点和表情结果。

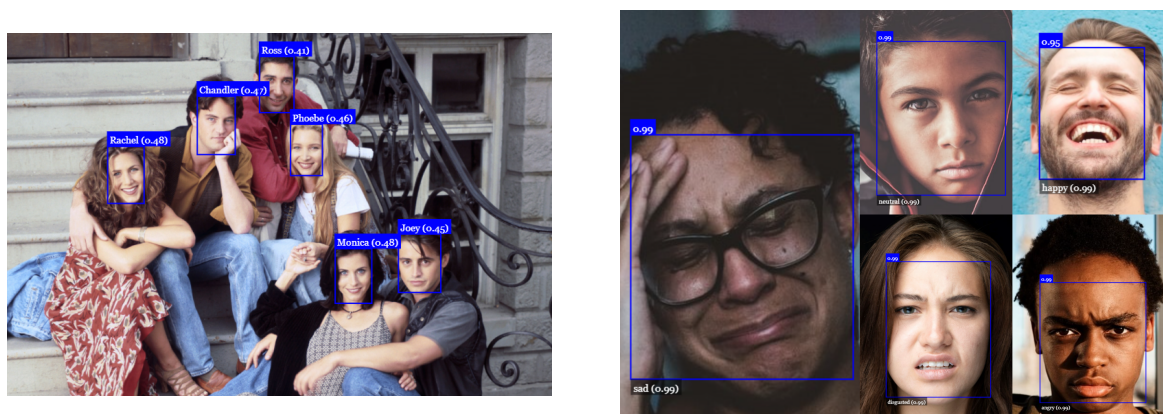


图 1.9 人脸检测与综合演示效果

从工程实现看，项目具有较好的演示性。页面入口清楚，模型资源保存在本地，业务脚本集中在 `dist/`，示例图片和人物图片保存在 `public/`。这种组织方式方便教师讲解，也方便学生对照代码与报告复现实验。相比只展示最终效果，报告中分析

文件结构和脚本流程更能体现课程实验的完整性。

1.11.2 性能影响因素

浏览器端推理的性能主要受三个因素影响。第一是模型加载时间。首次打开页面时，浏览器需要读取多个模型权重文件，模型越多，初始等待越明显。第二是单次推理时间。图片越大、人脸越多、模型越复杂，推理时间越长。第三是页面渲染成本。Canvas 绘制和 DOM 更新虽然不属于神经网络计算，但在实时任务中也会影响流畅度。

摄像头页面还受到设备摄像头分辨率、CPU/GPU 能力、浏览器实现和当前系统负载影响。脚本通过 Tiny Face Detector、100 ms 检测间隔和侧边栏节流来控制开销，但并不能保证所有设备都有相同效果。如果输入视频分辨率较高或设备性能较弱，实时标注可能出现延迟、掉帧或结果抖动。

1.11.3 可靠性与数据质量

人脸分析结果对输入质量依赖明显。正脸、光照均匀、面部无遮挡的图像通常更容易得到稳定结果；侧脸、模糊、强背光、表情幅度小、多人人脸重叠或遮挡严重的图像可能导致漏检、误检或属性估计偏差。对于摄像头场景，用户移动、光照变化和镜头自动曝光也会影响每一帧结果。

因此，实验报告不应只写“系统能够识别”，还应写清它在什么条件下更容易识别、在什么条件下可能失败。这样的分析能体现对模型应用边界的理解。尤其是年龄、性别和表情等属性识别，输出本身带有概率性和不确定性，不适合脱离实验语境做确定判断。

1.12 隐私、安全与改进方向

摄像头功能涉及人脸和视频流等敏感信息。当前项目在浏览器端本地处理数据，没有主动把视频流上传到服务器，这是浏览器端部署的一个隐私优势。但是，只要页面请求摄像头权限，就应该让用户理解授权含义。课程报告中应说明：实验只用于本地演示，不应保存、传播或公开未经同意的人脸图像。

从工程改进角度看，项目还可以继续优化。第一，可以抽取公共逻辑，例如模型加载、空状态渲染、检测结果排序、Canvas 尺寸同步等，减少三个脚本之间的重复。第二，可以加入更明确的加载进度和错误提示，例如模型文件缺失、图片格式不支持、摄像头权限拒绝等。第三，可以构造定量测试集，统计不同光照、姿态、遮挡和多人脸场景下的检测成功率、属性稳定性和实时帧率。

从实验写作角度看，后续还可以补充更多真实运行截图。图片页面截图能证明上传和标注流程正常，摄像头截图能证明实时追踪有效，目录结构截图能证明模型和资源均来自本地仓库。相比只放模型示例图，真实运行截图更能体现课程项目的复现过程。

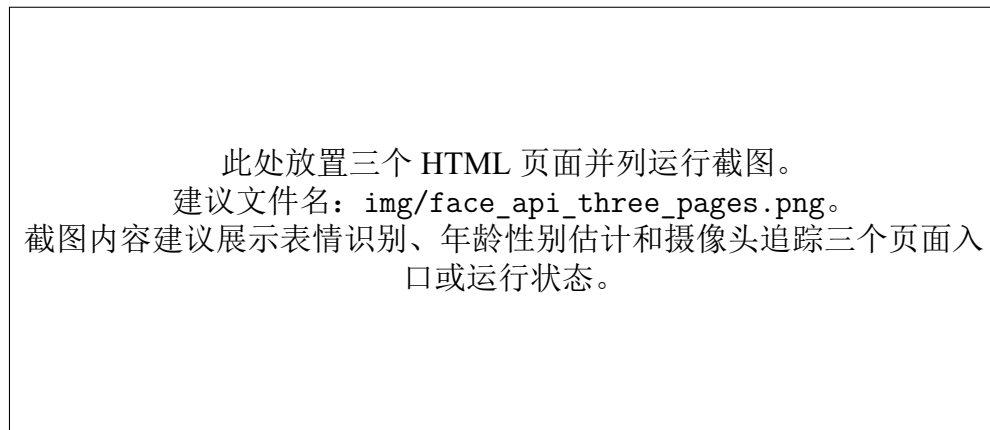


图 1.10 三个浏览器实验页面运行状态占位图

1.13 本章小结

本章围绕 `face-recognition` 项目分析了浏览器端人脸分析实验。实验展示了静态网页如何利用 `face-api.js` 加载本地预训练模型，并在图片和摄像头视频流两类输入上完成人脸检测、关键点定位、表情识别以及年龄性别估计。章节重点放在前端推理阶段，包括页面结构、模型加载、输入转换、`Canvas` 绘制、结果渲染和状态管理。

通过三个模块的对比可以看出，浏览器端深度学习推理适合课程演示、交互原型和轻量级体验场景。它降低了后端部署成本，也减少了敏感图像上传的需求。但它仍然受到设备性能、浏览器环境、输入质量和模型偏差影响。下一章将从 `dlib` 开源示例出发，进一步分析人脸描述子模型在训练阶段如何通过深度度量学习获得可用于距离匹配的特征空间。

2 dlib 开源人脸识别实例分析

2.1 分析目的与章节边界

上一章分析了 `face-api.js` 如何在浏览器端加载预训练模型并完成推理。本章换一个角度，分析 `dlib` 官方示例 `code/dlib/examples/dnn_metric_learning_on_images_ex.cpp`，重点说明人脸识别模型背后的深度度量学习思想。两章之间的关系

不是“同一项目的前后端实现”，而是“推理应用”和“训练思想”的对应：浏览器页面展示模型如何被调用，dlib 示例展示类似的人脸描述子模型可以如何训练。

本章主体不再重复前端页面的上传、Canvas 绘制和结果渲染流程，而是围绕 C++ 示例中的训练数据组织、mini-batch 采样、网络结构、损失函数、训练器、多线程加载、模型保存和阈值验证展开。只在开头和结尾保留少量对照，帮助理解为什么浏览器项目中的 和注册样本并不等同于重新训练模型。

在人脸识别任务中，系统真正要解决的问题通常不是把输入图像分到有限类别中，而是判断两张或多张人脸是否属于同一身份。如果直接把每个人作为一个分类类别，模型输出层就与训练集身份数量绑定。后续新增人物时，系统往往需要重新训练或微调分类器。度量学习采用不同思路：模型学习一个向量空间，使同一身份的人脸向量距离更近，不同身份的人脸向量距离更远。这样，新人物只需要登记若干参考图片并保存对应特征向量，识别时比较距离即可。

dlib 示例的价值正在于此。它没有把人脸识别写成普通分类任务，而是使用 训练图像嵌入模型。网络最后输出 128 维向量，这个向量就是人脸描述子。描述子本身不是姓名，也不是类别编号，而是人脸在特征空间中的位置。识别阶段只需要计算向量距离，并用阈值判断相似或不相似。

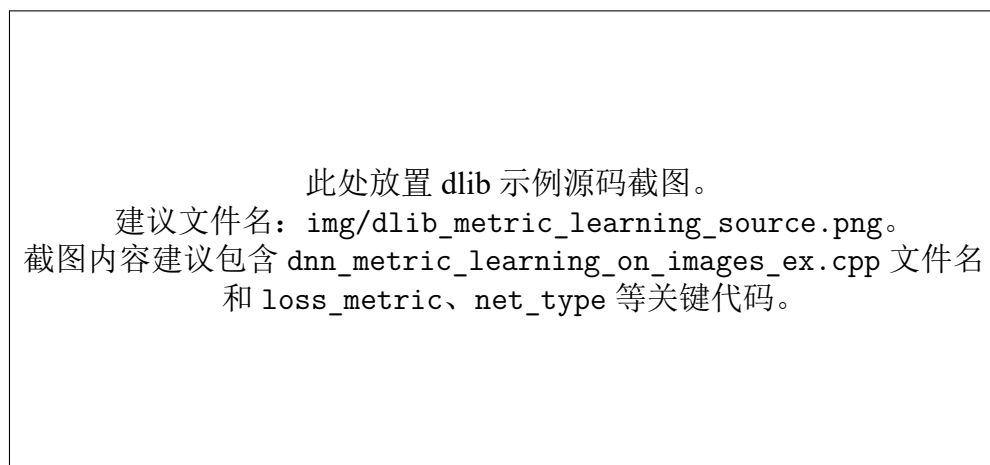


图 2.1 dlib 度量学习示例源码占位图

2.2 示例入口与数据目录组织

2.2.1 命令行输入与顶层目录

dlib 示例要求命令行传入一个顶层目录。该目录下的每个子目录代表一个身份，子目录中放置该身份的多张图片。示例注释中的结构可以概括为：

```
1 top_level_directory/  
2   person1/  
3     image1.jpg  
4     image2.jpg  
5   person2/  
6     image3.jpg  
7     image4.jpg
```

这种结构把身份标签编码在目录层级中。程序并不关心人物目录的具体名称,也不要求图片文件名遵循统一格式;它只利用“同一子目录中的图片属于同一身份”这一事实生成训练标签。这样的数据组织方式简单直观,也便于人工检查数据是否放错目录。

示例中的 会遍历顶层目录下的所有子目录,并把每个子目录中的图片路径收集成一个。最终得到的数据结构是:外层向量表示不同身份,内层向量表示某个身份对应的图片列表。后续 mini-batch 采样会直接基于这个结构随机选择身份和图片。

此处放置训练数据目录截图。
建议文件名: `img/dlib_training_dataset_tree.png`。
截图内容建议展示顶层目录、多个身份子目录和每个身份下的多张图片。

图 2.2 dlib 训练数据目录组织占位图

2.2.2 训练数据与参考样本的区别

dlib 示例中的目录用于训练网络参数。程序会反复读取这些图片,生成 mini-batch, 计算损失并更新网络权重。浏览器项目中的 `public/img/labeled_images` 虽然也按人物目录组织,但作用不同:它只用于提取参考描述子并构造匹配器,不会更新预训练模型。因此,两者在目录形态上相似,在训练语义上不同。

这个区别很重要。如果把浏览器项目中的注册图片误解为训练集,就会误以为页面可以通过几张图片“训练出”新模型。实际上,页面只是利用已经训练好的描述子网络,把参考图片转换成向量,再把新图片向量与参考向量做距离比较。dlib 示

例则展示了描述子网络本身如何通过大量有标签图片训练出来。

2.3 mini-batch 采样机制

2.3.1 为什么度量学习依赖 batch 组成

普通分类任务的 mini-batch 只需要包含图片和类别标签，模型通过分类损失学习每张图片属于哪个类别。度量学习则更依赖 batch 内部的配对关系。一次训练步骤中，损失函数需要同时看到“应该靠近的样本对”和“应该远离的样本对”。如果 batch 中每个身份只有一张图片，就无法形成同身份正样本对；如果 batch 中身份数量太少，又缺少足够的负样本对。

dlib 示例通过 明确控制 batch 组成。函数参数 表示一次采样多少个不同身份，表示每个身份采样多少张图片。主训练流程调用，即每次选择 5 个身份，每个身份选 5 张图片，共得到 25 张图片。这样，同一身份内部任意两张图片构成正样本对，不同身份之间的图片构成负样本对。

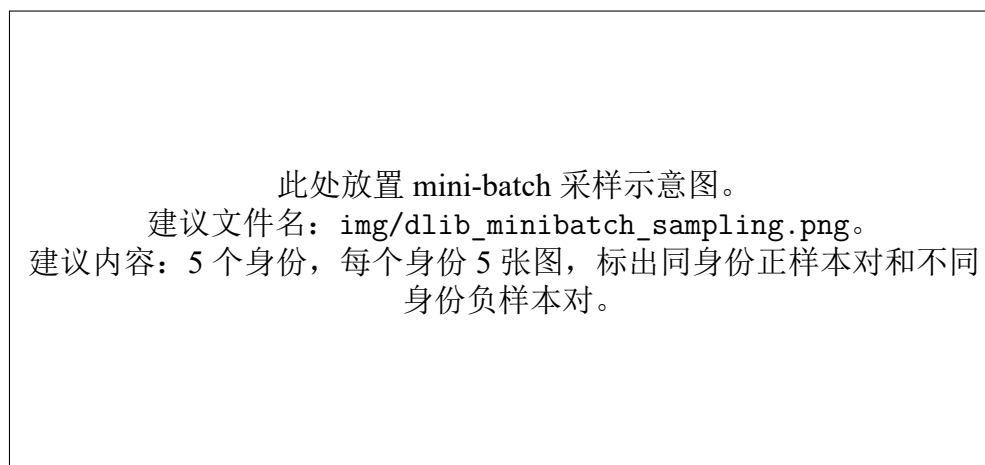


图 2.3 深度度量学习 mini-batch 采样占位图

2.3.2 随机选择与标签生成

首先清空 和 ，然后用 避免同一个身份在一次 batch 中被重复选中。对每个被选中的身份，函数再从该身份图片列表中随机选择 张图片，调用 读取图像，并把身份编号写入 。

这里的标签是身份在外层向量中的下标，而不是人物姓名字符串。对于训练损失来说，标签只需要表达“相同”或“不同”的关系。只要两张图片标签相同，损失函数就会把它们当作同身份样本；标签不同，就会当作不同身份样本。也就是说，度量学习并不关心类别名称的语义，只关心样本之间的关系。

2.3.3 数据增强与尺寸约束

采样完成后，示例会对图片进行简单数据增强。代码先调用 `扰动颜色`，再以较高概率调用 `对图像做轻微变换`。颜色扰动可以模拟亮度、色彩和拍摄环境变化，抖动变换可以让模型对局部位置变化更鲁棒。对于人脸识别而言，同一身份可能出现在不同光照、角度和成像条件下，数据增强有助于模型学习更稳定的身份特征。

示例还强调 `batch` 中所有图片尺寸必须一致。深度网络的输入张量需要统一高度、宽度和通道数。如果训练集中图片尺寸差异较大，通常要提前裁剪、缩放或对齐。`dlib` 示例没有把完整的人脸检测对齐流程写入本文件，而是假设输入图片已经适合训练。真实项目中，训练前的数据清洗、裁剪和对齐往往会占据大量工作。

2.4 网络结构与 128 维描述子

2.4.1 ResNet 主体结构

示例中的网络结构来自 `dlib` 的 `ResNet` 示例，并将普通分类损失替换为 `triplet_loss`。代码使用大量类型别名定义残差块和网络层级，例如 `resnet_block`、`resnet_stage` 到 `resnet_network`。网络前端包含卷积、`ReLU` 和池化操作，中间由多级残差结构提取特征，末端通过全局平均池化和全连接层输出固定长度向量。

残差结构的意义在于缓解深层网络训练困难。通过跳跃连接，网络可以学习残差映射，而不是每一层都从零学习完整变换。对于图像识别任务，残差网络能够在较深层级提取从边缘、纹理到面部结构的多层特征。`dlib` 示例的网络比大型图像分类网络更小，但仍保留了 `ResNet` 的核心思想。

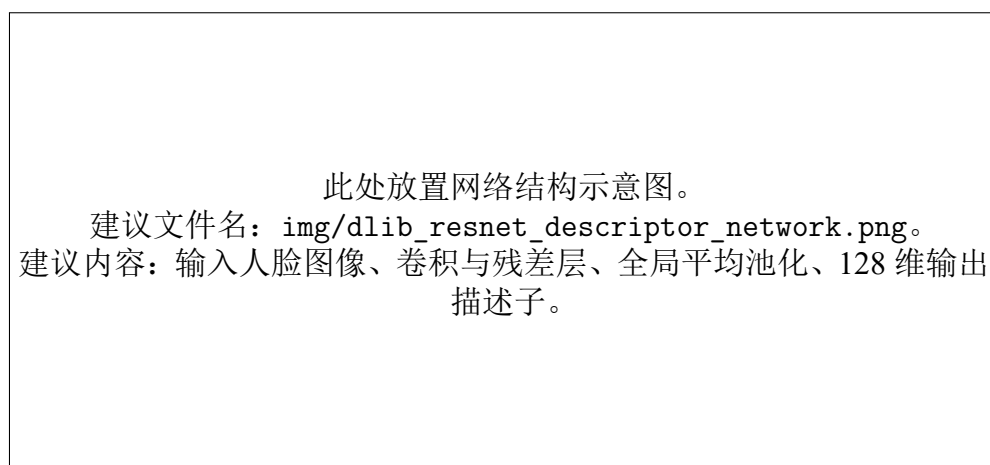


图 2.4 `dlib` 人脸描述子网络结构占位图

2.4.2 训练网络与测试网络

示例定义了两个网络类型：和。用于训练，网络内部使用批归一化层；用于测试和推理，把批归一化替换为固定仿射变换。这样做是因为训练阶段可以使用 batch 统计量，而推理阶段通常希望网络行为固定，不再依赖当前 batch 的均值和方差。

这种训练网络与推理网络的区分体现了深度学习工程中的常见做法。模型训练时为了稳定梯度和加速收敛，可以使用批归一化；模型部署时则需要把训练阶段学到的统计量固化下来，使单张图片或小 batch 输入也能得到稳定输出。浏览器端预训练模型虽然不直接暴露这些 C++ 类型，但同样经历了从训练形态到推理形态的转换。

2.4.3 128 维输出向量

网络末端的表示输出 128 维向量，表示在进入全连接层前做全局平均池化。这个 128 维向量就是人脸描述子。描述子不是分类概率，也不是姓名，而是模型为输入人脸生成的紧凑特征表示。

使用固定长度描述子有两个好处。第一，它便于比较。任意两张人脸图片经过同一个网络后都会得到 128 维向量，可以直接计算欧氏距离。第二，它支持开放集识别。系统不需要预先固定所有可能人物类别，只要为新人物保存若干参考描述子，就能通过最近邻或阈值比较进行匹配。这也是它比普通闭集分类更适合人脸识别应用的原因。

2.5 loss_metric 损失函数

2.5.1 距离关系作为训练目标

的作用是把样本之间的距离关系写入训练目标。训练时，网络根据标签知道哪些图片属于同一身份，哪些图片属于不同身份。对于同身份样本，损失函数希望它们在嵌入空间中的距离小；对于不同身份样本，损失函数希望它们的距离足够大。随着训练进行，网络逐渐学会把身份相关特征编码进 128 维向量。

这种目标与普通 softmax 分类不同。softmax 分类学习的是“输入属于哪个类别”，通常输出固定类别上的概率分布；度量学习学习的是“输入之间应该如何排列”，输出的是可比较的特征向量。对于人脸识别场景，后者更灵活，因为实际系统经常需要新增从未出现在训练集中的身份。

2.5.2 嵌入空间的直观理解

可以把训练后的嵌入空间想象成一个高维坐标系。每张人脸图片经过网络后都会落在这个坐标系中的某个点。同一个人的多张图片虽然拍摄角度、表情和光照不同，但应该落在相近区域；不同人的图片应该落在相对远的区域。识别时，只要比较两个点之间的距离，就可以判断它们是否可能属于同一身份。

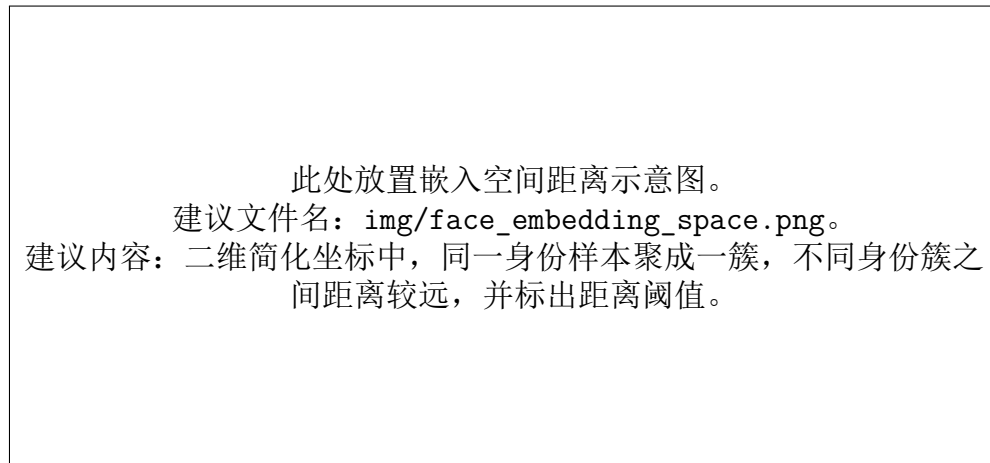


图 2.5 人脸嵌入空间与距离阈值占位图

需要强调的是，实际描述子是 128 维，无法直接画在二维平面上。占位图中的二维图只是一种教学示意，用于说明“同类靠近、异类远离”的思想。真实模型中的距离关系由训练数据、网络结构和损失函数共同决定。

2.6 训练器与优化过程

2.6.1 训练器配置

主函数中先构造 `Trainer`，再创建 `Trainer`。训练器使用随机梯度下降优化网络参数，其中 `optimizer` 和 `device` 分别对应优化器中的权重衰减和动量设置。随后，代码调用 `Trainer.train` 设置初始学习率，并调用 `Trainer.train` 输出训练日志。

示例还调用 `Trainer.save_checkpoint` 定期保存同步文件。同步文件的作用是让训练过程具备一定恢复能力。如果训练耗时很长，中途异常中断时，可以利用同步状态继续训练，而不是完全从头开始。这体现了实际训练工程对可靠性的要求。

2.6.2 学习率停止条件

训练循环的停止条件是学习率降到 `1e-5` 以下。示例中设置这个阈值是为了让示例程序可以较快结束；如果要训练高质量模型，通常需要更充分的数据、更长训练时间和更细致的学习率策略。学习率下降意味着优化过程进入后期，参数更新幅度变小，模型逐渐接近当前设置下的收敛状态。

训练循环本身不断从数据队列中取出图片和标签，然后调用 `optimizer.step()` 完成一次训练步骤。每一步都会前向计算描述子，依据标签计算距离损失，再反向传播更新网络参数。这个过程反复进行，直到学习率降低到停止阈值。

2.7 主函数训练流程拆解

2.7.1 参数检查与数据读入

示例的 `main` 函数首先检查命令行参数数量。如果没有传入训练数据目录，程序会输出用法说明并结束。这说明训练程序把数据路径作为外部输入，而不是写死在源码中。随后，程序调用 `load_data` 读取顶层目录，并输出 `num_images`，用于确认当前数据集中包含多少个身份。

这一输出虽然简单，但在实验复现中很有价值。如果 `num_images` 为 0 或明显小于预期，说明训练目录结构不正确，可能是路径传错、子目录为空，或者图片没有被识别为有效文件。训练前先检查身份数量，可以避免程序进入后续训练后才发现数据无效。

2.7.2 训练对象初始化

读取数据后，程序构造 `net` 作为待训练网络，再创建 `optimizer`。训练器绑定到网络对象，因此后续所有梯度更新都会作用在 `net` 上。随后，程序设置学习率、日志输出和同步文件。这些配置共同定义了训练过程的外部行为：学习率决定更新步幅，日志用于观察训练进度，同步文件用于恢复。

从工程角度看，这一阶段相当于训练任务的初始化。只有网络、优化器、学习率、数据列表和同步策略都准备好，程序才真正进入训练循环。课程报告中把这一阶段写清楚，可以帮助读者理解训练程序并不是一上来就“训练模型”，而是先建立数据和优化环境。

2.7.3 生产者消费者训练循环

主训练循环没有直接调用 `optimizer.step()`，而是从 `data_loader` 和 `net` 两个管道中取数据。数据加载线程负责生产 `batch`，训练主线程负责消费 `batch`。核心步骤可以概括为：加载线程读取图片并生成标签，放入队列；训练线程从队列取出图片和标签；训练器执行 `optimizer.step()` 更新网络参数。

这种结构把 I/O 和计算分离。对于大规模图片训练，如果没有数据加载队列，训练线程可能频繁等待磁盘和 CPU 图像解码；有了队列之后，只要加载速度足够快，训练线程就能持续获得 `batch`。示例虽然是教学代码，但已经体现了真实训练系统常用的流水线思想。

此处放置 dlib 主函数训练流程图。
建议文件名：img/dlib_main_training_flow.png。
建议内容：参数检查、读取数据、初始化网络、启动加载线程、训练循环、保存模型、验证阈值。

图 2.6 dlib 示例主函数训练流程占位图

2.8 多线程数据加载

2.8.1 为什么需要异步加载

深度学习训练不只消耗 GPU 计算，也消耗 CPU 读取、解码和增强图片的能力。如果训练线程每次都等待磁盘读取和图像处理，GPU 可能处于空闲状态。为了提高吞吐量，dlib 示例使用 和多个数据加载线程，让数据加载与训练计算并行进行。

示例创建了两个管道：用于传递图片 batch，用于传递标签 batch。数据加载线程不断调用 生成图片和标签，再把它们放入队列；训练主线程从队列中取出 batch 并执行训练。这个模式可以概括为“加载线程生产数据，训练线程消费数据”。

2.8.2 线程数量与资源平衡

示例启动了 5 个数据加载线程，并在注释中提示线程数量应根据 CPU 核心数调整。线程过少可能无法及时提供数据，导致训练等待；线程过多则可能造成 CPU 竞争、内存占用增加和上下文切换开销。真实训练任务需要根据数据集大小、磁盘速度、CPU 核心数和 GPU 性能进行平衡。

这部分代码说明，训练一个人脸描述子模型并不是只有网络结构和损失函数。数据加载、增强、队列缓冲、线程同步和异常处理都会影响训练效率。相比之下，浏览器项目只执行推理，不承担这些训练工程复杂度，因此前端页面显得更轻量。

2.9 训练数据准备与质量控制

2.9.1 身份数量与样本数量

度量学习模型的效果高度依赖训练数据。首先，身份数量必须足够多。如果只有少量身份，模型很难学习到具有泛化能力的身份区分特征。其次，每个身份需要

此处放置训练日志或多线程加载示意截图。
建议文件名：`img/dlib_training_log.png`。
截图内容建议包含训练日志输出、学习率变化或数据加载队列示意。

图 2.7 dlib 训练日志与数据加载占位图

多张图片。如果每个身份只有一张图，模型无法学习同一身份在不同姿态、表情和光照下的变化范围。示例中每个 `batch` 采样 5 个身份、每个身份 5 张图片，正是为了在一次训练步骤中同时呈现身份内变化和身份间差异。

训练数据还需要避免严重不均衡。如果某些身份图片很多，而另一些身份只有一两张，随机采样可能让模型更多看到样本丰富的身份。示例按身份随机选择，再按身份内图片随机选择，可以在一定程度上控制 `batch` 中身份分布，但数据集本身仍应尽量保证每个身份有足够样本。

2.9.2 图片质量与预处理

训练图片最好满足清晰、对齐、主体明确和尺寸一致等条件。如果图片中人脸位置差异过大，模型可能把背景、姿态或裁剪方式当成身份特征，从而降低泛化能力。真实训练流程通常会先用人脸检测器找到人脸区域，再根据关键点进行对齐和裁剪，使眼睛、鼻子、嘴巴等结构落在相对稳定的位置。

`dlib` 示例关注度量学习训练本身，没有完整展开人脸检测和对齐流程。因此，在课程报告中应说明：该示例适理解训练机制，但真实人脸识别系统还需要前置数据清洗。数据清洗包括删除非人脸图片、剔除错误身份目录、统一图像尺寸、处理重复图片、检查严重模糊和遮挡样本等。

2.9.3 数据增强的作用边界

数据增强可以提升模型鲁棒性，但不能替代真实数据多样性。颜色扰动和图像抖动能模拟一定范围内的拍摄变化，但如果训练集中缺少侧脸、不同年龄、不同肤色、不同光照或不同设备来源的数据，模型仍可能在这些场景中表现较差。增强方法只能在已有数据分布附近扩展样本，不能凭空创造完整的现实变化。

因此，训练数据准备应同时关注数量、质量和覆盖范围。对于课程实验，可以

不实际训练大模型，但报告中要认识到：模型性能不是由网络结构单独决定的，数据本身同样决定了描述子空间能否泛化到新图片。

2.10 模型保存与训练后验证

2.10.1 模型清理与序列化

训练结束后，程序调用 `wait_for_training_thread` 等待训练线程状态同步，然后输出训练完成信息。接着，代码调用 `clear_training_state` 清理训练过程中不再需要的临时状态，最后执行 `save_network` 将网络保存到磁盘。保存后的文件可以在后续程序中反序列化加载，用于提取人脸图像的 128 维描述子。

这里需要注意文件名中的 `renset` 拼写来自示例代码本身。报告分析时应保持与源码一致，不主动改成 `resnet`，否则读者对照代码时会困惑。模型保存步骤标志着训练阶段产物形成，后续应用阶段只需加载该产物执行推理。

2.10.2 训练后验证流程

示例最后没有只停留在保存模型，而是继续演示如何使用训练好的网络。程序重新采样一个 `mini-batch`，将训练网络赋值给测试网络，然后调用 `get_all_image_embeddings` 得到所有图片的嵌入向量。接着，程序两两比较向量距离，统计判断正确的样本对数量。

如果两张图片标签相同，它们的向量距离应该小于阈值；如果标签不同，距离应该大于或等于阈值。这个验证流程虽然比较简单，但清楚展示了度量学习模型的使用方式：模型负责输出描述子，判别逻辑通过距离和阈值完成。

2.11 距离阈值与开放集识别

2.11.1 阈值来源

示例使用 `distance_threshold` 取得距离阈值。这个阈值来自 `distance_constraint` 的距离约束。训练过程中，损失函数会围绕这个距离目标调整向量空间，使同身份样本倾向于落在阈值以内，不同身份样本倾向于落在阈值以外。

阈值判断是人脸识别系统的重要环节。如果阈值设置过大，不同身份可能被误判为同一人；如果阈值设置过小，同一身份在姿态、光照变化较大时可能被误判为不同人。实际应用通常需要在验证集上评估误识率和拒识率，再根据应用场景选择合适阈值。安全要求高的场景往往倾向于更严格阈值，便利性要求高的场景可能接受更宽松阈值。

2.11.2 与闭集分类的区别

开放集识别意味着测试阶段可能出现训练阶段没有见过的人。度量学习天然适合这种场景，因为模型输出的是通用描述子，而不是固定类别概率。新增人物时，只需要采集若干参考图片，提取描述子并存入数据库；识别时，将新图片描述子与数据库中的参考描述子比较即可。

闭集分类则假设所有可能类别都在训练集中。它适合固定类别任务，例如识别有限种物体；但在人脸识别中，新增身份非常常见。如果每新增一个人都重新训练输出层，工程成本会很高。因此，dlib 示例展示的描述子加距离匹配思想，更符合人脸识别系统的扩展需求。

2.12 验证指标与阈值选择

2.12.1 num_right 与 num_wrong 的含义

示例在训练后验证阶段定义了 `num_right` 和 `num_wrong` 两个计数器。程序遍历当前 `mini-batch` 中所有图片对，如果两张图片标签相同且距离小于阈值，则计为正确；如果两张图片标签不同且距离大于或等于阈值，也计为正确。相反，同身份距离过大或不同身份距离过小都会计为错误。

这个验证方式可以直观展示描述子空间是否符合训练目标。若 `num_right` 较多、`num_wrong` 较少，说明当前 `batch` 上同身份与不同身份的距离关系较好。不过，这只是对一个随机 `mini-batch` 的简单检查，不能代替完整测试集评估。真正可靠的模型评估需要在独立数据集上统计更多指标。

2.12.2 正负样本对的数量

在一次 `batch` 中，共有 25 张图片。每个身份 5 张图片，因此每个身份内部可以形成 $5 \times 4 / 2 = 10$ 个正样本对，5 个身份共形成 50 个正样本对。不同身份之间的负样本对数量更多：任意两个身份之间有 $5 \times 5 = 25$ 对，5 个身份两两组合共有 $5 \times 4 / 2 = 10$ 组，因此负样本对共 250 对。

这说明度量学习 `batch` 中负样本对通常远多于正样本对。损失函数需要在大量负样本中找到对训练有帮助的关系，同时保持同身份样本紧密。实际训练中，样本挖掘策略、`batch` 组成和阈值设置都会影响训练效果。示例采用较简单的随机采样方式，适合教学理解。

2.12.3 阈值调节的取舍

阈值选择本质上是在误识和拒识之间取舍。误识是把不同身份判断为同一人，拒识是把同一身份判断为不同人。降低阈值通常会减少误识，但增加拒识；提高阈值通常会减少拒识，但增加误识。不同应用对两类错误的容忍度不同，因此不能只说某个阈值“最好”，而应结合应用场景选择。

例如，在门禁或支付等安全场景中，误识可能造成更严重后果，因此阈值应更严格；在照片聚类或相册整理场景中，偶尔把同一人拆成两个簇的代价较低，可以使用相对宽松的策略，再允许用户手动修正。课程实验中应重点说明阈值的作用和取舍，而不是把某个固定阈值视为普遍适用。

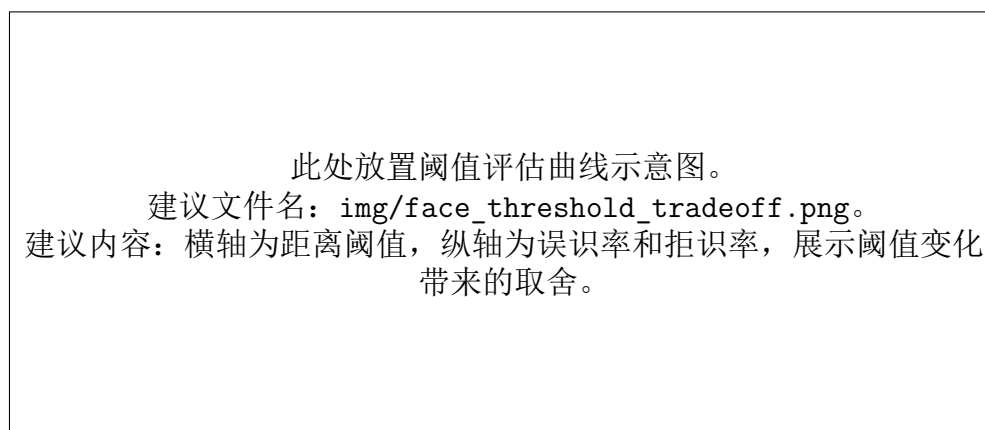


图 2.8 距离阈值选择与错误类型取舍占位图

2.13 与浏览器实验的对应关系

dlib 示例可以帮助澄清浏览器实验中容易混淆的几个概念。第一，训练数据用于更新网络参数，参考样本用于登记待识别人物；第二，模型输出的人脸描述子是向量，不是姓名；第三，识别结果来自距离比较，而不是模型直接输出人物名称；第四，训练阶段需要处理 mini-batch、数据增强、损失函数、学习率和模型保存，推理阶段则主要处理模型加载、输入转换、描述子提取和结果展示。

浏览器项目中的 可以理解为度量学习思想在推理端的简化使用。它并不训练网络，而是接收若干 ，在新图片描述子到来时计算距离并返回最接近的标签。dlib 示例则说明这些描述子为什么具有可比较性：因为训练阶段已经通过 让同身份向量靠近、不同身份向量远离。

因此，两章合起来可以形成完整链路：第 2 章说明模型如何通过深度度量学习获得特征空间，第 1 章说明前端页面如何加载预训练模型并在浏览器中执行推理。为了保持章节边界，本章只在这里做对照，不再展开前端页面的 DOM、Canvas 和摄

像头流程。

2.14 实验启示与局限

dlib 示例虽然展示了完整训练框架，但它仍然是教学示例，而不是可直接用于生产的人脸识别系统。真实系统需要更大规模、更干净、更均衡的数据集，需要严格的人脸检测、对齐和质量控制流程，也需要在独立验证集上评估模型性能。训练后的描述子模型还需要考虑跨年龄、跨光照、跨设备、跨姿态和遮挡条件下的泛化能力。

此外，人脸识别具有明显的隐私和安全属性。训练数据必须来自合法、合规、授权的来源；模型应用也应限制在明确告知和必要范围内。课程报告分析算法流程时，也应避免把技术可行性等同于应用合理性。特别是身份识别系统，一旦用于门禁、考勤、支付或风控等场景，就必须考虑误识、拒识、偏差、数据泄露和用户权利等问题。

从学习角度看，dlib 示例最值得借鉴的是方法论：用目录组织身份数据，用 mini-batch 同时提供正负样本关系，用 ResNet 提取图像特征，用 约束嵌入距离，用阈值完成训练后验证。这些步骤构成了人脸描述子模型训练的基本骨架。理解这个骨架后，再看浏览器端的预训练模型推理，就能更清楚地区分“模型已经学到的能力”和“页面临时完成的匹配逻辑”。

2.15 本章小结

本章围绕 dlib 官方示例 `code/dlib/examples/dnn_metric_learning_on_images_ex.cpp`，分析了深度度量学习在人脸识别中的训练流程。示例通过按身份组织图片、构造包含多个身份和多张图片的 mini-batch、使用数据增强提升鲁棒性、采用 ResNet 主体网络输出 128 维描述子，并通过 约束同身份和不同身份样本之间的距离。

训练完成后，模型被序列化保存，并可以通过描述子距离和阈值进行验证。这种“描述子提取 + 距离判断”的思想解释了人脸识别系统为什么不必把每个人都作为固定分类类别，也解释了浏览器实验中注册样本和 的作用。与上一章结合后，报告能够从训练思想和前端推理两个层面说明人脸分析系统的整体工作方式。

参考文献

- [1] Vincent Muhler. face-api.js: JavaScript API for face detection and face recognition in the browser and nodejs with tensorflow.js. GitHub, 2018.
- [2] Vincent Muhler. face-api.js model weights. GitHub repository.
- [3] Davis E. King. Dlib-ml: A Machine Learning Toolkit. Journal of Machine Learning Research, 2009.
- [4] Davis E. King. dlib C++ Library. GitHub repository.
- [5] World Wide Web Consortium. Media Capture and Streams. W3C Recommendation.
- [6] Bootstrap Team. Bootstrap Documentation.

附录 人脸分析实验关键代码

本附录列出浏览器端人脸分析实验中与功能实现直接相关的页面结构和业务脚本片段。为避免重复展示第三方库源码，附录不收录 `face-api.min.js`，只展示页面如何引入模型库、承载输入输出区域，以及三个实验模块如何完成模型加载、图片推理、结果渲染和摄像头实时追踪。

页面入口与展示容器

图片表情识别页面在文档头部延迟加载 `face-api.min.js` 和当前页面的业务脚本。主体区域左侧提供图片上传控件和识别结果容器，右侧的 `analysis-stage` 用于插入用户上传的图片和覆盖绘制结果的画布。

这里保留脚本标签的原因是，代码引入顺序直接影响页面能否运行。`face-api.min.js` 必须先于业务脚本加载，业务脚本才能访问 `Object`；`defer` 则保证脚本在 DOM 解析完成后按顺序执行，使业务脚本能够拿到上传控件、展示区域等真实节点。

```
21 <script defer src="dist/face-api.min.js"></script>
22 <script defer src="dist/expression_script.js"></script>

75 <section class="panel control-panel">
76   <h2 class="panel-heading"> 上传图片 </h2>
77   <input id="upload" type="file" accept="image/*" />
78   <label for="upload" class="upload-dropzone">
79     <span class="upload-label"> 选择图片 </span>
80     <span class="upload-hint"> 建议使用清晰、正脸较明显的照片。</span>
81   </label>
82   <p class="panel-note"> 支持常见 JPG、PNG 图片格式。</p>
83 </section>
84
85 <section class="panel info-panel">
86   <h2 class="panel-heading"> 识别结果 </h2>
87   <div id="expression-results" class="result-summary">
88     <div class="result-empty">
89       <strong> 等待识别 </strong>
90       <span> 上传图片后，这里会同步显示表情识别结果副本。</span>
91     </div>
92   </div>
93 </section>
94 </aside>
95
96 <section class="panel result-panel">
```

```

97     <div id="analysis-stage" class="analysis-stage">
98         <div class="stage-empty" data-empty-state>
99             <strong> 等待图片 </strong>
100             <span> 上传后在这里显示表情识别结果。</span>
101         </div>

```

年龄与性别估计页面的入口结构与表情识别页面基本一致，只是业务脚本替换为 `dist/estimation_script.js`，结果容器也变为 `estimation-results`。这种“页面结构相同、业务脚本不同”的组织方式有利于复用样式和交互习惯。

```

21 <script defer src="dist/face-api.min.js"></script>
22 <script defer src="dist/estimation_script.js"></script>

```

```

75 <section class="panel control-panel">
76     <h2 class="panel-heading"> 上传图片 </h2>
77     <input id="upload" type="file" accept="image/*" />
78     <label for="upload" class="upload-dropzone">
79         <span class="upload-label"> 选择图片 </span>
80         <span class="upload-hint"> 建议使用清晰、无遮挡的人像照片。</span>
81     </label>
82     <p class="panel-note"> 支持常见 JPG、PNG 图片格式。</p>
83 </section>
84
85 <section class="panel info-panel">
86     <h2 class="panel-heading"> 识别结果 </h2>
87     <div id="estimation-results" class="result-summary">
88         <div class="result-empty">
89             <strong> 等待识别 </strong>
90             <span> 上传图片后，这里会同步显示年龄与性别估计结果。</span>
91         </div>
92     </div>
93 </section>
94 </aside>
95
96 <section class="panel result-panel">
97     <div id="analysis-stage" class="analysis-stage">
98         <div class="stage-empty" data-empty-state>
99             <strong> 等待图片 </strong>
100             <span> 上传后在这里显示年龄与性别估计结果。</span>

```

摄像头实时追踪页面同样通过脚本标签加载模型库和业务逻辑。页面中的追踪区域 tracking-stage 用来叠加视频与检测画布。

video 元素负责承接浏览器摄像头视频流，并为后续逐帧识别提供输入。

```
21 <script defer src="dist/face-api.min.js"></script>
22 <script defer src="dist/webcam_script.js"></script>

101 <section class="panel tracking-panel">
102   <div id="tracking-stage" class="tracking-stage">
103     <div class="stage-empty" data-empty-state>
104       <strong> 等待摄像头权限 </strong>
105       <span> 授权后在这里显示实时画面与识别结果。</span>
106     </div>
107     <video
108       id="video"
109       width="1280"
110       height="720"
111       autoplay
112       muted
113       playsinline
114     ></video>
115   </div>
```

图片表情识别脚本

表情识别脚本首先取得页面中的上传控件、展示区域和结果区域，并定义本地模型路径。随后通过 并行加载检测、关键点、人脸描述子、表情识别和 SSD MobileNetV1 模型，全部模型加载完成后才进入 流程。

```
1  const imageUpload = document.getElementById("upload");
2  const analysisStage = document.getElementById("analysis-stage");
3  const expressionResults = document.getElementById("expression-results");
4  const MODEL_URL = "./models";
5
6  const EXPRESSION_LABELS = {
7    neutral: " 中性",
8    happy: " 高兴",
9    sad: " 难过",
10   angry: " 生气",
11   fearful: " 害怕",
12   disgusted: " 厌恶",
```

```
13     surprised: "惊讶",
14   };
15   const EXPRESSION_ORDER = Object.keys(EXPRESSION_LABELS);
16
17   Promise.all([
18     faceapi.nets.tinyFaceDetector.loadFromUri(MODEL_URL),
19     faceapi.nets.faceLandmark68Net.loadFromUri(MODEL_URL),
20     faceapi.nets.faceRecognitionNet.loadFromUri(MODEL_URL),
21     faceapi.nets.faceExpressionNet.loadFromUri(MODEL_URL),
22     faceapi.nets.ssdMobilenetv1.loadFromUri(MODEL_URL),
23   ]).then(start);
```

这一段代码体现了两个关键点。第一，指向 `./models`，说明权重从项目本地目录加载，不依赖远程网络。第二，模型加载使用 `Promise.all`，所有异步加载任务都完成后才调用。如果缺少这一步等待，用户上传图片时模型可能尚未准备好，推理调用会失败。

为了让模型输出更容易阅读，脚本将 `face-api.js` 返回的英文表情标签映射为中文标签，并按检测框横坐标排序多张人脸。结果渲染函数会选出概率最高的主表情，同时把七类表情概率渲染为进度条，便于观察模型对不同表情类别的置信度分布。

```
38 function getSortedDetections(detections) {
39   return [...detections].sort(
40     (first, second) => first.detection.box.x - second.detection.box.x
41   );
42 }
43
44 function getTopExpression(expressions) {
45   return EXPRESSION_ORDER.reduce(
46     (currentTop, key) => {
47       const score = expressions[key] || 0;
48       return score > currentTop.score ? { key, score } : currentTop;
49     },
50     { key: EXPRESSION_ORDER[0], score: expressions[EXPRESSION_ORDER[0]] || 0 }
51   );
52 }
53
54 function renderExpressionResults(detections) {
55   const sortedDetections = getSortedDetections(detections);
56
57   if (!sortedDetections.length) {
58     renderEmptyState(
59       expressionResults,
```



```
60     " 未检测到人脸",
61     " 请尝试上传清晰、正脸更明显的照片。"
62 );
63 return;
64 }
65
66 expressionResults.innerHTML = sortedDetections
67   .map((detection, index) => {
68     const topExpression = getTopExpression(detection.expressions);
69     const metrics = EXPRESSION_ORDER.map((key) => {
70       const score = detection.expressions[key] || 0;
71
72       return `
73         <li class="result-metric">
74           <div class="result-metric__row">
75             <span class="result-metric__label">${EXPRESSION_LABELS[key]}</span>
76             <span class="result-metric__value">${formatPercent(score)}</span>
77           </div>
78           <div class="result-bar"><span style="width: ${Math.round(
79             score * 100
80           )}%"></span></div>
81         </li>
82       `;
83     }).join("");
84
85     return `
86       <article class="result-card">
87         <div class="result-card__header">
88           <span class="result-card__title"> 人脸 ${index + 1}</span>
89           <span class="status-pill">${EXPRESSION_LABELS[topExpression.key]}</span>
90         </div>
91         <p class="result-card__highlight">
92           主表情: <strong>${EXPRESSION_LABELS[topExpression.key]}</strong>
93           (${formatPercent(topExpression.score)})
94         </p>
95         <ul class="result-list">${metrics}</ul>
96       </article>
97     `;
98   })
99   .join("");
100 }
```

图片上传监听器是该模块的核心入口。用户选择文件后，脚本先清理旧图片和旧画布，再将文件转换为浏览器图片对象，创建覆盖画布并匹配尺寸。随后，脚本依次执行人脸检测、关键点定位、人脸描述子提取和表情识别，最后将结果缩放到展示尺寸并绘制检测框和表情标签。

```
102  async function start() {
103      const container = document.createElement("div");
104      container.className = "analysis-stage__media";
105      analysisStage.append(container);
106
107      let image;
108      let canvas;
109
110      imageUpload.addEventListener("change", async () => {
111          if (!imageUpload.files.length) {
112              return;
113          }
114
115          renderEmptyState(expressionResults, " 正在识别", " 图片分析中，请稍候。");
116
117          if (image) {
118              image.remove();
119          }
120          if (canvas) {
121              canvas.remove();
122          }
123
124          image = await faceapi.bufferToImage(imageUpload.files[0]);
125          container.append(image);
126
127          canvas = faceapi.createCanvasFromMedia(image);
128          container.append(canvas);
129
130          const displaySize = { width: image.width, height: image.height };
131          faceapi.matchDimensions(canvas, displaySize);
132
133          const detections = await faceapi
134              .detectAllFaces(image)
135              .withFaceLandmarks()
136              .withFaceDescriptors()
137              .withFaceExpressions();
138      });
```

```
139     const resizedDetections = faceapi.resizeResults(detections, displaySize);
140     faceapi.draw.drawDetections(canvas, resizedDetections);
141     faceapi.draw.drawFaceExpressions(canvas, resizedDetections);
142     renderExpressionResults(resizedDetections);
143
144     analysisStage.classList.add("stage-ready");
145   });
146 }
```

其中 负责把本地文件对象转换成浏览器可绘制的图片元素， 创建与图片对应的覆盖画布， 保证画布坐标系与图片显示尺寸一致。链式调用 表示先检测所有人脸，再补充关键点、人脸描述子和表情结果。最后的 用于把模型输出坐标映射回展示尺寸。

年龄与性别估计脚本

年龄与性别估计脚本与表情识别脚本结构相近，但加载的任务模型不同。它除了检测、关键点和描述子模型外，还加载 ，用于输出估计年龄、性别标签和性别分类置信度。

```
1  const imageUpload = document.getElementById("upload");
2  const analysisStage = document.getElementById("analysis-stage");
3  const estimationResults = document.getElementById("estimation-results");
4  const MODEL_URL = "./models";
5
6  const GENDER_LABELS = {
7    male: " 男性",
8    female: " 女性",
9  };
10
11  Promise.all([
12    faceapi.nets.tinyFaceDetector.loadFromUri(MODEL_URL),
13    faceapi.nets.faceLandmark68Net.loadFromUri(MODEL_URL),
14    faceapi.nets.faceRecognitionNet.loadFromUri(MODEL_URL),
15    faceapi.nets.ageGenderNet.loadFromUri(MODEL_URL),
16    faceapi.nets.ssdMobilenetv1.loadFromUri(MODEL_URL),
17  ]).then(start);
```

这一模块同样使用本地 `models` 目录。与表情识别相比，它把任务网络从 换成 。这说明页面的基础流程可以复用：只要输入仍然是人脸区域，后续可以在同一检测结果上接入不同属性估计网络。

结果渲染部分把模型返回的 和 映射为中文展示文本，并将年龄估计值四舍五入后写入结果卡片。性别置信度继续用百分比和进度条表示，使侧边栏结果能够与右侧检测框形成对照。

```
38 function renderEstimationResults(detections) {
39   const sortedDetections = getSortedDetections(detections);
40
41   if (!sortedDetections.length) {
42     renderEmptyState(
43       estimationResults,
44       " 未检测到人脸",
45       " 请尝试上传清晰、无遮挡的人像照片。"
46     );
47     return;
48   }
49
50   estimationResults.innerHTML = sortedDetections
51     .map((detection, index) => {
52       const gender = detection.gender || "male";
53       const genderLabel = GENDER_LABELS[gender] || gender;
54       const confidence = detection.genderProbability || 0;
55
56       return `
57         <article class="result-card">
58           <div class="result-card__header">
59             <span class="result-card__title"> 人脸 ${index + 1}</span>
60             <span class="status-pill">${genderLabel}</span>
61           </div>
62           <div class="result-detail-grid">
63             <div class="result-detail">
64               <span class="result-detail__label"> 年龄估计 </span>
65               <span class="result-detail__value">${Math.round(
66                 detection.age
67               )} 岁 </span>
68             </div>
69             <div class="result-detail">
70               <span class="result-detail__label"> 性别判断 </span>
71               <span class="result-detail__value">${genderLabel}</span>
72             </div>
73           </div>
74           <ul class="result-list">
75             <li class="result-metric">
76               <div class="result-metric__row">
```

```
77         <span class="result-metric__label"> 性别置信度 </span>
78         <span class="result-metric__value">${formatPercent(
79             confidence
80         )}</span>
81     </div>
82     <div class="result-bar"><span style="width: ${Math.round(
83         confidence * 100
84     )}%"></span></div>
85 </li>
86 </ul>
87 </article>
88 `;
89 })
90 .join("");
91 }
```

图片推理流程仍然从上传事件开始。与表情识别不同的是，链式调用最后使用 `renderEmptyState`，并在画布上额外通过 `image.remove()` 绘制包含年龄和性别的检测框标签。

```
93 async function start() {
94     const container = document.createElement("div");
95     container.className = "analysis-stage__media";
96     analysisStage.append(container);
97
98     let image;
99     let canvas;
100
101     imageUpload.addEventListener("change", async () => {
102         if (!imageUpload.files.length) {
103             return;
104         }
105
106         renderEmptyState(estimationResults, " 正在识别", " 图片分析中，请稍候。");
107
108         if (image) {
109             image.remove();
110         }
111         if (canvas) {
112             canvas.remove();
113         }
114
115         image = await faceapi.bufferToImage(imageUpload.files[0]);
```

```
116     container.append(image);
117
118     canvas = faceapi.createCanvasFromMedia(image);
119     container.append(canvas);
120
121     const displaySize = { width: image.width, height: image.height };
122     faceapi.matchDimensions(canvas, displaySize);
123
124     const detections = await faceapi
125       .detectAllFaces(image)
126       .withFaceLandmarks()
127       .withFaceDescriptors()
128       .withAgeAndGender();
129
130     const resizedDetections = faceapi.resizeResults(detections, displaySize);
131     faceapi.draw.drawDetections(canvas, resizedDetections);
132
133     resizedDetections.forEach((detection, index) => {
134       const box = resizedDetections[index].detection.box;
135       const drawBox = new faceapi.draw.DrawBox(box, {
136         label: `${Math.round(detection.age)} years old ${detection.gender}`,
137       });
138       drawBox.draw(canvas);
139     });
140
141     renderEstimationResults(resizedDetections);
142
143     analysisStage.classList.add("stage-ready");
144   });
145 }
```

从代码结构看，年龄与性别估计和表情识别最大的差异只在两处：一是模型加载列表不同，二是链式推理末端不同。其余的文件读取、图片插入、画布创建、尺寸匹配、检测结果缩放等流程完全相似。这也说明前端人脸分析页面适合按“基础检测流程 + 任务网络”的方式扩展。

摄像头实时追踪脚本

摄像头脚本在页面初始化时保存视频元素、追踪区域、状态区和结果区引用，并维护覆盖画布、定时器和结果刷新时间等状态。模型加载完成后进入，为后续实时识别做准备。

```
1  const video = document.getElementById("video");
2  const trackingStage = document.getElementById("tracking-stage");
3  const webcamStatus = document.getElementById("webcam-status");
4  const webcamResults = document.getElementById("webcam-results");
5  const MODEL_URL = "./models";
6  let canvas;
7  let detectionInterval;
8  let lastResultsMarkup = "";
9  let lastResultsRenderAt = 0;
10
11  const EXPRESSION_LABELS = {
12    neutral: "中性",
13    happy: "高兴",
14    sad: "难过",
15    angry: "生气",
16    fearful: "害怕",
17    disgusted: "厌恶",
18    surprised: "惊讶",
19  };
20  const EXPRESSION_ORDER = Object.keys(EXPRESSION_LABELS);
21  const SIDEBAR_REFRESH_MS = 280;
22
23  Promise.all([
24    faceapi.nets.tinyFaceDetector.loadFromUri(MODEL_URL),
25    faceapi.nets.faceLandmark68Net.loadFromUri(MODEL_URL),
26    faceapi.nets.faceRecognitionNet.loadFromUri(MODEL_URL),
27    faceapi.nets.faceExpressionNet.loadFromUri(MODEL_URL),
28  ]).then(startVideo);
29
30  function getDisplaySize() {
31    return {
32      width: Math.max(trackingStage.clientWidth, 1),
33      height: Math.max(trackingStage.clientHeight, 1),
34    };
35  }
36
37  function syncCanvasSize() {
38    if (!canvas) {
39      return getDisplaySize();
40    }
41
42    const displaySize = getDisplaySize();
```

```
43
44   if (canvas.width !== displaySize.width || canvas.height !== displaySize.height) {
45       faceapi.matchDimensions(canvas, displaySize);
46   }
47
48   return displaySize;
49 }
```

与图片页面不同，摄像头页面需要维护跨帧状态。保存覆盖画布，保存定时器编号，和 用于控制侧边栏刷新频率。这些变量保证页面在连续视频输入下不会不断创建新画布或过度更新 DOM。

实时页面需要频繁更新侧边栏，因此脚本把结果 HTML 的生成和渲染拆开处理。渲染函数会比较本次结果与上次结果，并使用 控制刷新闻隔，避免每 100 ms 都重复写入 DOM。

```
87 function getTopExpression(expressions) {
88     return EXPRESSION_ORDER.reduce(
89         (currentTop, key) => {
90             const score = expressions[key] || 0;
91             return score > currentTop.score ? { key, score } : currentTop;
92         },
93         { key: EXPRESSION_ORDER[0], score: expressions[EXPRESSION_ORDER[0]] || 0 }
94     );
95 }
96
97 function buildResultsMarkup(detections) {
98     const sortedDetections = getSortedDetections(detections);
99
100    if (!sortedDetections.length) {
101        return `
102            <div class="result-empty">
103                <strong> 未检测到人脸 </strong>
104                <span> 当前画面中还没有检测到清晰人脸。</span>
105            </div>
106        `;
107    }
108
109    return sortedDetections
110        .map((detection, index) => {
111            const topExpression = getTopExpression(detection.expressions);
112            const metrics = EXPRESSION_ORDER.map((key) => {
```



```

113     const score = detection.expressions[key] || 0;
114
115     return `
116         <li class="result-metric">
117             <div class="result-metric__row">
118                 <span class="result-metric__label">${EXPRESSION_LABELS[key]}</span>
119                 <span class="result-metric__value">${formatPercent(score)}</span>
120             </div>
121             <div class="result-bar"><span style="width: ${Math.round(
122                 score * 100
123             )}%"></span></div>
124         </li>
125     `;
126     }).join("");
127
128     return `
129     <article class="result-card">
130         <div class="result-card__header">
131             <span class="result-card__title"> 人脸 ${index + 1}</span>
132             <span class="status-pill">${EXPRESSION_LABELS[topExpression.key]}</span>
133         </div>
134         <p class="result-card__highlight">
135             主表情: <strong>${EXPRESSION_LABELS[topExpression.key]}</strong>
136             (${formatPercent(topExpression.score)})
137         </p>
138         <ul class="result-list">${metrics}</ul>
139     </article>
140     `;
141     })
142     .join("");
143 }
144
145 function renderWebcamResults(detections, force = false) {
146     const markup = buildResultsMarkup(detections);
147     const now = Date.now();
148
149     if (!force && markup === lastResultsMarkup) {
150         return;
151     }
152
153     if (!force && now - lastResultsRenderAt < SIDEBAR_REFRESH_MS) {
154         return;
155     }

```

```
156
157   webcamResults.innerHTML = markup;
158   lastResultsMarkup = markup;
159   lastResultsRenderAt = now;
160 }
```

摄像头启动函数兼容了现代 和旧版浏览器前缀接口。脚本会先显示等待授权状态，在用户允许访问摄像头后把视频流写入；如果浏览器不支持或授权失败，则更新状态区和结果区提示。

```
162 function startVideo() {
163   renderStatus(" 等待授权", " 未连接", " 允许浏览器访问摄像头后开始实时识别。");
164   renderEmptyResults(
165     " 等待识别",
166     " 摄像头开始工作后，这里会同步显示表情识别结果副本。",
167     true
168   );
169
170   const legacyGetUserMedia =
171     navigator.getUserMedia ||
172     navigator.webkitGetUserMedia ||
173     navigator.mozGetUserMedia;
174
175   const getUserMedia =
176     navigator.mediaDevices && navigator.mediaDevices.getUserMedia
177     ? (constraints) => navigator.mediaDevices.getUserMedia(constraints)
178     : legacyGetUserMedia
179     ? (constraints) =>
180       new Promise((resolve, reject) => {
181         legacyGetUserMedia.call(navigator, constraints, resolve, reject);
182       })
183     : null;
184
185   if (!getUserMedia) {
186     console.error("Camera API is not supported in this browser.");
187     renderStatus(" 摄像头不可用", " 不支持", " 当前浏览器不支持摄像头访问接口。");
188     renderEmptyResults(
189       " 无法开始识别",
190       " 请更换支持摄像头访问的浏览器后重试。",
191       true
192     );
193     return;
```

```
194 }
195
196 renderStatus(" 正在连接", " 连接中", " 正在请求摄像头权限, 请稍候。");
197
198 getUserMedia({ video: {} })
199   .then((stream) => {
200     video.srcObject = stream;
201     renderStatus(" 摄像头已连接", " 已连接", " 画面已连接, 等待浏览器开始播放视频流。");
202   })
203   .catch((error) => {
204     console.error(error);
205     renderStatus(
206       " 访问失败",
207       " 不可用",
208       " 未能获取摄像头权限, 或者当前设备不可用。"
209     );
210     renderEmptyResults(
211       " 无法开始识别",
212       " 请检查浏览器授权和摄像头设备后重试。",
213       true
214     );
215   });
216 }
```

当视频开始播放时, 脚本创建覆盖画布并启动定时器。每轮定时任务都会同步画布尺寸, 对当前视频帧执行人脸检测、关键点定位和表情识别, 再清空上一帧标注并重绘最新检测框、关键点和表情标签。视频暂停或结束时, 脚本及时清除定时器, 避免后台继续执行无意义的推理。

```
218 video.addEventListener("play", () => {
219   if (!canvas) {
220     canvas = faceapi.createCanvasFromMedia(video);
221     trackingStage.append(canvas);
222   }
223
224   syncCanvasSize();
225   trackingStage.classList.add("stage-live");
226   renderStatus(" 正在识别", " 识别中", " 已连接摄像头, 正在同步更新实时识别结果。");
227
228   window.clearInterval(detectionInterval);
229   detectionInterval = window.setInterval(async () => {
```

```
230     const displaySize = syncCanvasSize();
231     const detections = await faceapi
232       .detectAllFaces(video, new faceapi.TinyFaceDetectorOptions())
233       .withFaceLandmarks()
234       .withFaceExpressions();
235
236     const resizedDetections = faceapi.resizeResults(detections, displaySize);
237     canvas.getContext("2d").clearRect(0, 0, canvas.width, canvas.height);
238     faceapi.draw.drawDetections(canvas, resizedDetections);
239     faceapi.draw.drawFaceLandmarks(canvas, resizedDetections);
240     faceapi.draw.drawFaceExpressions(canvas, resizedDetections);
241     renderWebcamResults(resizedDetections);
242   }, 100);
243 });
244
245 window.addEventListener("resize", () => {
246   syncCanvasSize();
247 });
248
249 ["pause", "ended"].forEach((eventName) => {
250   video.addEventListener(eventName, () => {
251     window.clearInterval(detectionInterval);
252     renderStatus(" 识别已暂停", " 已暂停", " 摄像头画面已暂停, 左侧结果已停止更新。");
253     renderEmptyResults(
254       " 识别已暂停",
255       " 恢复摄像头画面后, 这里会重新同步显示识别结果。",
256       true
257     );
258   });
259 });
```

这段代码中的 `100` 表明实时检测采用轻量级检测器。每 100 ms 执行一次推理大致相当于每秒最多 10 次检测，实际帧率还会受到设备性能、浏览器和摄像头分辨率影响。定时器中先调用 `clearRect` 清除上一帧绘制内容，再绘制新一帧结果，否则旧检测框会残留在画布上。

开源人脸识别模型训练代码

本项目网页端直接使用 `face-api.js` 的预训练权重，并没有在浏览器项目中重新训练深度网络。为了说明人脸识别模型的训练思想，本节引用已经克隆到 `code/dlib/` 目录中的 `dlib` 开源训练示例 `dnn_metric_learning_on_images_ex.cpp`。该示例展示了

如何使用 训练人脸嵌入模型，使同一身份的人脸向量距离更近，不同身份的人脸向量距离更远。

示例开头说明了深度度量学习在人脸识别中的作用：模型不直接把图像分类为固定身份，而是学习一个向量空间，再用距离或最近邻方法完成识别。

```
1  // The contents of this file are in the public domain. See  
   ↳ LICENSE_FOR_EXAMPLE_PROGRAMS.txt  
2  /*  
3     This is an example illustrating the use of the deep learning tools from the  
4     dlib C++ Library. In it, we will show how to use the loss_metric layer to do  
5     metric learning on images.  
6  
7     The main reason you might want to use this kind of algorithm is because you  
8     would like to use a k-nearest neighbor classifier or similar algorithm, but  
9     you don't know a good way to calculate the distance between two things. A  
10    popular example would be face recognition. There are a whole lot of papers  
11    that train some kind of deep metric learning algorithm that embeds face  
12    images in some vector space where images of the same person are close to each  
13    other and images of different people are far apart. Then in that vector  
14    space it's very easy to do face recognition with some kind of k-nearest  
15    neighbor classifier.  
16  
17    In this example we will use a version of the ResNet network from the  
18    dnn_imagenet_ex.cpp example to learn to map images into some vector space where  
19    pictures of the same person are close and pictures of different people are far  
20    apart.  
21  
22    You might want to read the simpler introduction to the deep metric learning  
23    API, dnn_metric_learning_ex.cpp, before reading this example. You should  
24    also have read the examples that introduce the dlib DNN API before  
25    continuing. These are dnn_introduction_ex.cpp and dnn_introduction2_ex.cpp.  
26  
27    */
```

训练数据按身份目录组织。顶层目录下的每个子目录表示一个人物身份，子目录内保存该人物的多张图片。加载函数遍历这些子目录，形成按身份分组的图片列表，为后续 mini-batch 采样做准备。

```
38 // We will need to create some functions for loading data. This program will  
39 // expect to be given a directory structured as follows:  
40 //     top_level_directory/
```

```
41 //      person1/
42 //          image1.jpg
43 //          image2.jpg
44 //          image3.jpg
45 //      person2/
46 //          image4.jpg
47 //          image5.jpg
48 //          image6.jpg
49 //      person3/
50 //          image7.jpg
51 //          image8.jpg
52 //          image9.jpg
53 //
54 // The specific folder and image names don't matter, nor does the number of folders or
55 // images. What does matter is that there is a top level folder, which contains
56 // subfolders, and each subfolder contains images of a single person.
57
58 // This function spiders the top level directory and obtains a list of all the
59 // image files.
60 std::vector<std::vector<string>> load_objects_list (
61     const string& dir
62 )
63 {
64     std::vector<std::vector<string>> objects;
65     for (auto subdir : directory(dir).get_dirs())
66     {
67         std::vector<string> imgs;
68         for (auto img : subdir.get_files())
69             imgs.push_back(img);
70
71         if (imgs.size() != 0)
72             objects.push_back(imgs);
73     }
74     return objects;
75 }
```

mini-batch 采样函数继续从每个身份下随机选择图片，并生成与图片对应的整数标签。这个标签不是人物姓名，而是身份在外层向量中的索引。函数还进行了颜色扰动和图像抖动，并检查同一 batch 中图片尺寸是否一致。

```
77 // This function takes the output of load_objects_list() as input and randomly
78 // selects images for training. It should also be pointed out that it's really
```

```
79 // important that each mini-batch contain multiple images of each person. This
80 // is because the metric learning algorithm needs to consider pairs of images
81 // that should be close (i.e. images of the same person) as well as pairs of
82 // images that should be far apart (i.e. images of different people) during each
83 // training step.
84 void load_mini_batch (
85     const size_t num_people,    // how many different people to include
86     const size_t samples_per_id, // how many images per person to select.
87     dlib::rand& rnd,
88     const std::vector<std::vector<string>>& objs,
89     std::vector<matrix<rgb_pixel>>& images,
90     std::vector<unsigned long>& labels
91 )
92 {
93     images.clear();
94     labels.clear();
95     DLIB_CASSERT(num_people <= objs.size(), "The dataset doesn't have that many people in
96     ↪ it.");
97
98     std::vector<bool> already_selected(objs.size(), false);
99     matrix<rgb_pixel> image;
100     for (size_t i = 0; i < num_people; ++i)
101     {
102         size_t id = rnd.get_random_32bit_number()%objs.size();
103         // don't pick a person we already added to the mini-batch
104         while(already_selected[id])
105             id = rnd.get_random_32bit_number()%objs.size();
106         already_selected[id] = true;
107
108         for (size_t j = 0; j < samples_per_id; ++j)
109         {
110             const auto& obj = objs[id][rnd.get_random_32bit_number()%objs[id].size()];
111             load_image(image, obj);
112             images.push_back(std::move(image));
113             labels.push_back(id);
114         }
115     }
116
117     // You might want to do some data augmentation at this point. Here we do some simple
118     // color augmentation.
119     for (auto&& crop : images)
120     {
121         disturb_colors(crop, rnd);
122     }
123 }
```

```

121     // Jitter most crops
122     if (rnd.get_random_double() > 0.1)
123         crop = jitter_image(crop,rnd);
124 }
125
126
127 // All the images going into a mini-batch have to be the same size. And really, all
128 // the images in your entire training dataset should be the same size for what we are
129 // doing to make the most sense.
130 DLIB_CASSERT(images.size() > 0);
131 for (auto&& img : images)
132 {
133     DLIB_CASSERT(img.nr() == images[0].nr() && img.nc() == images[0].nc(),
134         "All the images in a single mini-batch must be the same size.");
135 }
136 }

```

网络结构部分定义了训练网络和测试网络。训练网络使用带批归一化的 ResNet 主体，输出层为，损失层为；测试网络则把批归一化替换为固定仿射变换，便于推理阶段稳定使用。

```

144 template <template <int,template<typename>class,int,typename> class block, int N,
    ↳ template<typename>class BN, typename SUBNET>
145 using residual = add_prev1<block<N,BN,1,tag1<SUBNET>>>>;
146
147 template <template <int,template<typename>class,int,typename> class block, int N,
    ↳ template<typename>class BN, typename SUBNET>
148 using residual_down =
    ↳ add_prev2<avg_pool<2,2,2,2,skip1<tag2<block<N,BN,2,tag1<SUBNET>>>>>>>>>;
149
150 template <int N, template <typename> class BN, int stride, typename SUBNET>
151 using block = BN<con<N,3,3,1,1,relu<BN<con<N,3,3,stride,stride,SUBNET>>>>>>>>;
152
153
154 template <int N, typename SUBNET> using res =
    ↳ relu<residual<block<N,bn_con,SUBNET>>>>;
155 template <int N, typename SUBNET> using ares =
    ↳ relu<residual<block<N,affine,SUBNET>>>>;
156 template <int N, typename SUBNET> using res_down =
    ↳ relu<residual_down<block<N,bn_con,SUBNET>>>>;
157 template <int N, typename SUBNET> using ares_down =
    ↳ relu<residual_down<block<N,affine,SUBNET>>>>;

```


158
159
160
161
162
163
164
165
166
167
168
169
170

171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194

训练阶段创建，设置学习率和同步文件，并通过多个加载线程持续产生 mini-batch。每个 mini-batch 包含多个身份以及每个身份的多张图片，使模型可以同时学

习同一身份和不同身份之间的距离关系。训练结束后，示例把网络清理并序列化保存为 `metric_network_renset.dat`。

训练器初始化代码如下。这里的 `sgd` 设置了权重衰减和动量，`0.1` 设置初始学习率，`std::chrono::minutes(5)` 则用于周期性同步训练状态。

```
218     net_type net;
219
220     dnn_trainer<net_type> trainer(net, sgd(0.0001, 0.9));
221     trainer.set_learning_rate(0.1);
222     trainer.be_verbose();
223     trainer.set_synchronization_file("face_metric_sync", std::chrono::minutes(5));
224     // I've set this to something really small to make the example terminate
225     // sooner. But when you really want to train a good model you should set
226     // this to something like 10000 so training doesn't terminate too early.
227     trainer.set_iterations_without_progress_threshold(300);
```

数据加载线程通过 `dlib::pipe` 把图片和标签送入训练主线程。主线程只需要不断取出队列中的 `batch` 并调用 `load_mini_batch`，从而把耗时的图片读取、解码和增强工作与训练计算并行起来。

```
229     // If you have a lot of data then it might not be reasonable to load it all
230     // into RAM. So you will need to be sure you are decompressing your images
231     // and loading them fast enough to keep the GPU occupied. I like to do this
232     // using the following coding pattern: create a bunch of threads that dump
233     // mini-batches into dlib::pipes.
234     dlib::pipe<std::vector<matrix<rgb_pixel>>> qimages(4);
235     dlib::pipe<std::vector<unsigned long>> qlabels(4);
236     auto data_loader = [&qimages, &qlabels, &objs](time_t seed)
237     {
238         dlib::rand rnd(time(0)+seed);
239         std::vector<matrix<rgb_pixel>> images;
240         std::vector<unsigned long> labels;
241         while(qimages.is_enabled())
242         {
243             try
244             {
245                 load_mini_batch(5, 5, rnd, objs, images, labels);
246                 qimages.enqueue(images);
247                 qlabels.enqueue(labels);
248             }
249             catch(std::exception& e)
250             {
```

```
251         cout << "EXCEPTION IN LOADING DATA" << endl;
252         cout << e.what() << endl;
253     }
254 }
255 };
256 // Run the data_loader from 5 threads. You should set the number of threads
257 // relative to the number of CPU cores you have.
258 std::thread data_loader1([data_loader]() { data_loader(1); });
259 std::thread data_loader2([data_loader]() { data_loader(2); });
260 std::thread data_loader3([data_loader]() { data_loader(3); });
261 std::thread data_loader4([data_loader]() { data_loader(4); });
262 std::thread data_loader5([data_loader]() { data_loader(5); });
263
264
265 // Here we do the training. We keep passing mini-batches to the trainer until the
266 // learning rate has dropped low enough.
267 while(trainer.get_learning_rate() >= 1e-4)
268 {
269     qimages.dequeue(images);
270     qlabels.dequeue(labels);
271     trainer.train_one_step(images, labels);
272 }
```

学习率降到阈值以下后，示例保存网络并关闭加载线程。保存模型前调用 是为了去掉训练时的临时缓存，使序列化文件更适合后续推理使用。

```
274 // Wait for training threads to stop
275 trainer.get_net();
276 cout << "done training" << endl;
277
278 // Save the network to disk
279 net.clean();
280 serialize("metric_network_renset.dat") << net;
281
282 // stop all the data loading threads and wait for them to terminate.
283 qimages.disable();
284 qlabels.disable();
285 data_loader1.join();
286 data_loader2.join();
287 data_loader3.join();
288 data_loader4.join();
289 data_loader5.join();
```

训练完成后，示例把图片送入测试网络得到嵌入向量，并根据向量距离判断两张图片是否属于同一身份。这个过程与浏览器项目中的 思路一致：先得到人脸描述子，再依据距离阈值完成匹配。

```
295 // Now, just to show an example of how you would use the network, let's check how
    ↪ well
296 // it performs on the training data.
297 dlib::rand rnd(time(0));
298 load_mini_batch(5, 5, rnd, objs, images, labels);
299
300 // Normally you would use the non-batch-normalized version of the network to do
301 // testing, which is what we do here.
302 anet_type testing_net = net;
303
304 // Run all the images through the network to get their vector embeddings.
305 std::vector<matrix<float,0,1>> embedded = testing_net(images);
306
307 // Now, check if the embedding puts images with the same labels near each other and
308 // images with different labels far apart.
309 int num_right = 0;
310 int num_wrong = 0;
311 for (size_t i = 0; i < embedded.size(); ++i)
312 {
313     for (size_t j = i+1; j < embedded.size(); ++j)
314     {
315         if (labels[i] == labels[j])
316         {
317             // The loss_metric layer will cause images with the same label to be less
318             // than net.loss_details().get_distance_threshold() distance from each
319             // other. So we can use that distance value as our testing threshold.
320             if (length(embedded[i]-embedded[j]) <
321                 ↪ testing_net.loss_details().get_distance_threshold())
322                 ++num_right;
323             else
324                 ++num_wrong;
325         }
326         else
327         {
328             if (length(embedded[i]-embedded[j]) >=
329                 ↪ testing_net.loss_details().get_distance_threshold())
330                 ++num_right;
331             else
332                 ++num_wrong;
333         }
334     }
335 }
```

```
331         }  
332     }  
333 }  
334  
335     cout << "num_right: " << num_right << endl;  
336     cout << "num_wrong: " << num_wrong << endl;  
337  
338 }
```

最后的两两比较代码尤其值得注意。它把同标签图片距离小于阈值视为正确,把不同标签图片距离大于等于阈值视为正确。这与浏览器端最近邻匹配的逻辑一致:模型负责输出描述子,识别逻辑负责比较描述子距离并决定是否接受某个身份标签。